

2026 비전공 교원 대상 AI 특강 (하계)

<기계학습 딥러닝 분야>

실습: 2026년 8월 5일 AutoML
소프트웨어학과 김광수 교수
인공지능학과 장승우 박사과정

AutoML

환경 설정

AutoML — 환경 설정

실습 환경 — 슈퍼컴퓨팅센터 GPU 클러스터

성균관대학교 슈퍼컴퓨팅센터 — 어떤 곳인가?

- 학교가 보유한 고성능 컴퓨팅(HPC) 자원을 교내·외 연구자에게 클라우드 형식으로 제공
- GPU 서버: NVIDIA A100(80GB)

오늘 우리는 — 이렇게 사용합니다

- 웹 브라우저로 접속 — 내 노트북에 아무것도 설치하지 않음
- 각자에게 도커 컨테이너 자동 배정
- 내 컴퓨터 사양과 무관 — 무거운 학습은 서버의 GPU가 대신 계산

구분	내부 사용자	외부 사용자
1 GPU	1,600원	3,200원

* 요금제 적용기간: 슈퍼컴퓨팅센터 운영위원회에서 결정

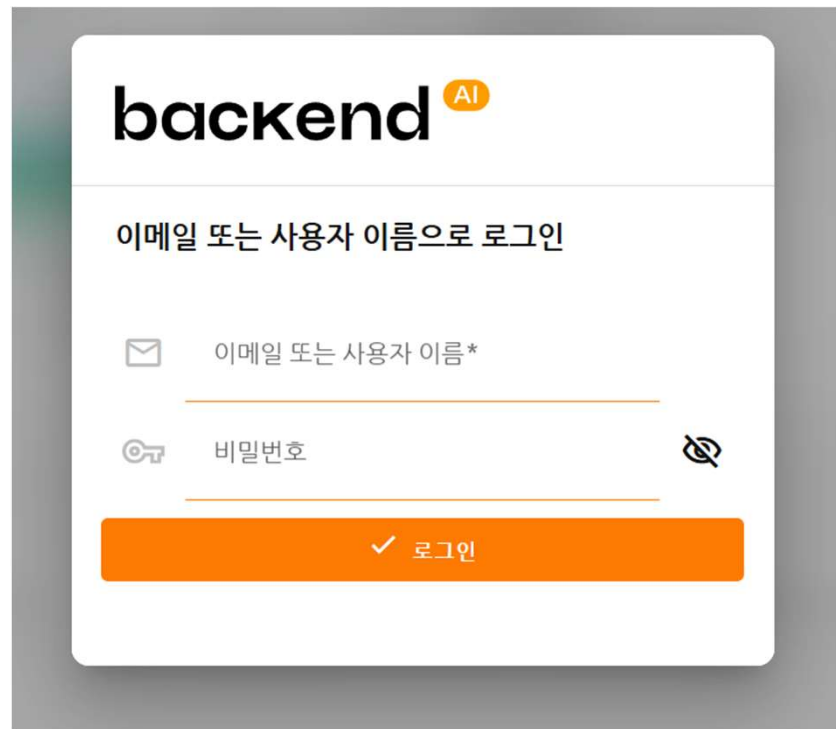
* 각 금액은 1시간 기준 요금임. 사용량 계산시 15분 단위 절사

* 통화는 KRW 기준이며, VAT미포함

<https://supercom.skku.edu/supercom/index.do>

AutoML — 환경 설정

실습 환경 — 슈퍼컴퓨팅센터 GPU 클러스터

A login form for 'backend AI'. The form has a title 'backend AI' with 'AI' in an orange circle. Below the title is the instruction '이메일 또는 사용자 이름으로 로그인'. There are two input fields: the first is labeled '이메일 또는 사용자 이름*' and the second is labeled '비밀번호' with a toggle icon. At the bottom is an orange button with a checkmark and the text '로그인'.

아이디: 본인의 SKKU 이메일 주소

비밀번호: 이메일 아이디의 @ 앞 부분 + !

sewo@skku.edu → sewo!

<https://spccluster.skku.edu/start>

실습 환경 — 슈퍼컴퓨팅센터 GPU 클러스터

2. VPN 접속(윈도우 PC, MAC, Mobile(Android, iOS))

- 1) <https://vpn.skku.edu:9000> 접속 (최초 접속 시 클라이언트 프로그램 다운로드)
- 2) 다운로드 된 클라이언트 프로그램 설치
- 3) 다운로드된 VPN 프로그램(AXGATE VPN) 실행
 - ☐ ID/PW : KINGO ID/PW로 로그인
- 4) OTP 등록(OS별 접속가이드 참조하여 구글 Authenticator 또는 AXGATE OTP 사용하여 QR 등록)
- 5) OTP 입력 후 사용

(※ VPN 사용은 윈도우 PC 사용을 권장합니다)

3. VPN 접속가이드(윈도우 PC, MAC, Mobile(Android, iOS), Linux)

- ☐ PC(Windows)용 접속 가이드 : [\[다운로드 링크\]](#) // [\[English Download\]](#)
- ☐ Mac용 접속 가이드 : [\[다운로드 링크\]](#)
- ☐ Android용 접속 가이드: [\[다운로드 링크\]](#)
- ☐ iOS용 접속 가이드: [\[다운로드 링크\]](#)
- ☐ Linux용 접속 가이드 : [\[다운로드 링크\]](#) / Linux용 접속 프로그램 : [\[다운로드 링크\]](#)
- ☐ OTP Registration Guide (Eng) : [\[Download\]](#)

<https://vpninfo.skku.edu/>

AutoML — 환경 설정

실습 환경 — 슈퍼컴퓨팅센터 GPU 클러스터

backend.ai WebUI

프로젝트 Edu

장승우

시작

대시보드

스토리지

데이터

워크로드

세션

나의 실행 환경

메트릭

자원 요약

통계

비밀번호를 변경해 주세요.
상단바의 사람 아이콘을 클릭하고 "사용자 정보 변경" 메뉴로 이동하세요.

시작 /

SKKU HPC Cluster [backend.ai]

-Pending status means 'entered the queue'

신규 최신 GPU(Blackwell) 연산 노드 하반기 도입 예정

새 스토리지 폴더 생성하기

폴더를 만들고 파일을 업로드합니다. 모델을 학습시키거나 외부 서비스를 제공하려면 해당 작업은 필수적입니다.

폴더 생성하기

Interactive 세션 생성

모델을 학습시킬 세션을 생성합니다. 원하는 환경과 리소스를 선택하여 코드를 실행하세요.

세션 시작

Batch 세션 생성

이미 정의된 파일 또는 예약된 작업에 대한 배치 세션을 생성합니다. 명령을 입력하고 날짜와 시간을 설정한 다음 필요에 따라 세션을 실행하세요.

세션 시작

URL 로 시작

GitHub, GitLab, Notebook 등 다양한 환경에서 프로젝트와 코드를 가져오고, 실행할 수 있습니다.

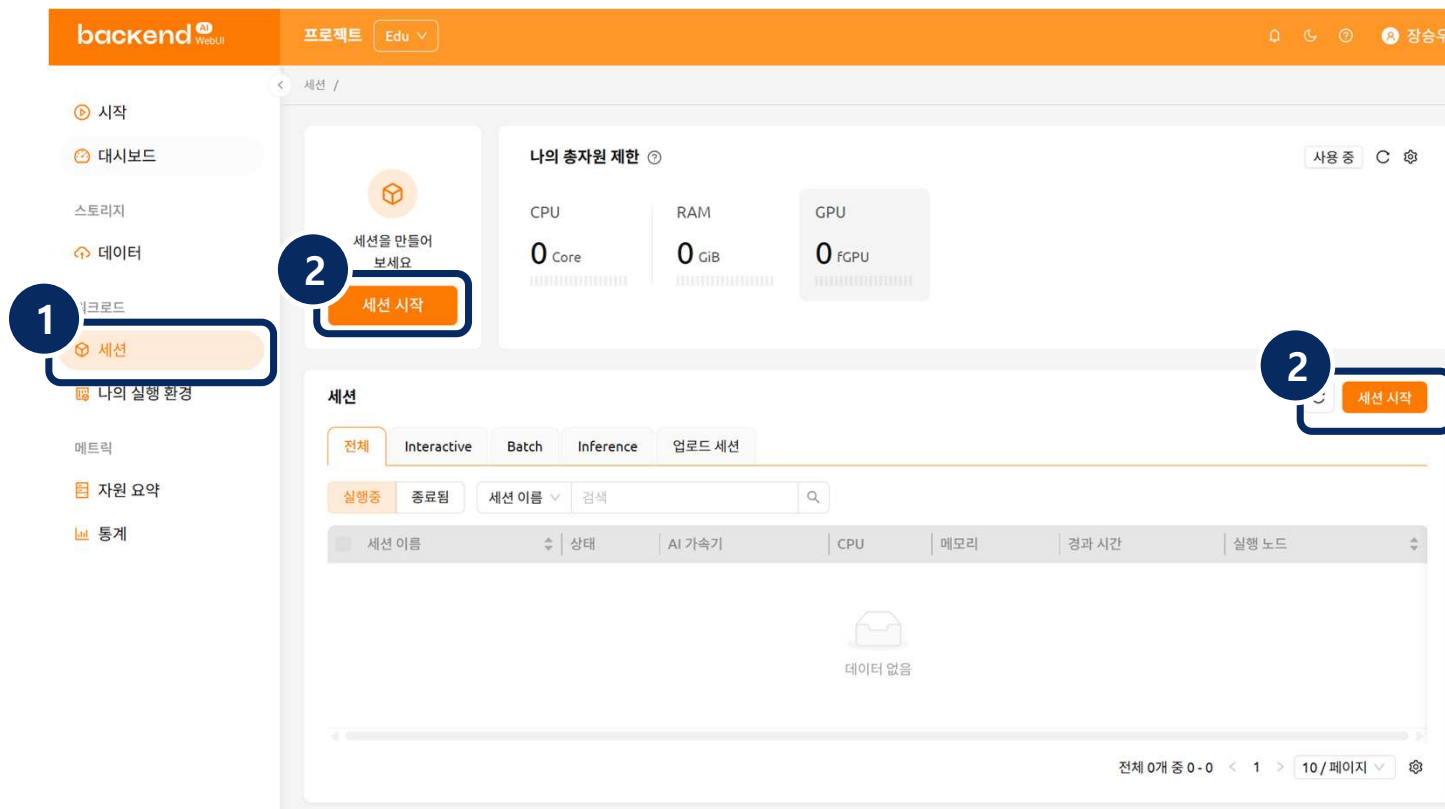
지금 시작하기

Applied AI & Computer Vision Lab · Sungkyunkwan University

6

AutoML — 환경 설정

실습 환경 — 슈퍼컴퓨팅센터 GPU 클러스터



실행 순서

- ① 왼쪽 메뉴에서 '세션' 클릭
- ② '세션 시작' 버튼 클릭

세션: 사용자가 모델 학습, 데이터 분석 등을 실행할 수 있는 독립적인 컴퓨팅 환경

AutoML — 환경 설정

실습 환경 — 세션 타입

새 세션 시작

최근 기록

1 세션 타입

2 실행 환경 & 자원 할당

3 데이터 & 폴더

4 네트워크

▶ 검토 및 시작

세션 타입

☒ **Interactive** 주피터 노트북, 비주얼 스튜디오 코드 등을 통해 코드를 인터랙티브하게 생성, 테스트, 실행할 수 있게 해줍니다.

☐ **Batch** 여러 노드 클러스터로 코드를 실행합니다.

세션 이름 (선택)

0 / 64

초기화

다음 >

검토로 건너뛰기 >>

↑
[다음] 클릭

AutoML — 환경 설정

실습 환경 — 실행 환경 & 자원 할당

새 세션 시작

최근 기록

실행 환경

실행 환경 / 버전 📄

PyTorch (NGC)

repo

NVIDIA GPU Cloud

▼

24.09

x86_64

PyTorch 2.5

Python 3.10

GPU: CUDA12.6

▼

환경 이름 (수동) (선택)

환경 변수 (선택)

+ 환경 변수 추가

AutoML — 환경 설정

실습 환경 — 실행 환경 & 자원 할당

자원 할당

자원 그룹

default

자원 프리셋

사용자 설정 자원 할당

CPU ⓘ

18

Core

1

60

메모리 ⓘ

MEM

48

GiB

0.625g

240g

Application MEM 47g

공유 메모리 (SHMEM) 1g 자동 ⓘ

AI 가속기 ⓘ

0.3

fGPU

0

1

자원 프리셋 – 사용자 설정 자원 할당

CPU – 18 Core

메모리 – 48 GiB

공유 메모리 (SHMEM) – 자동

AI 가속기 – 0.3 fGPU

AutoML — 환경 설정

실습 환경 — 데이터 & 폴더

스토리지

데이터

워크로드

세션

나의 실행 환경

메트릭

자원 요약

통계

폴더 생성 및 파일 업로드

폴더 생성

내 폴더

1 / 30

프로젝트 폴더

0

초대된 폴더

0

프로젝트 Edu

0%

사용자 장승우

0.22%

2.18 GB / 1 TB

폴더

C

폴더 생성

활성 1

휴지통

전체

일반

파이프라인

자동 마운트

모델

이름 ▾

검색

Q

☐	이름	제어	상태	위치	종류	마운트 권한	소유자
☐	SKKU		READY	sp-bai-sc-g010:sas	사용자	읽기 및 쓰기 R W	

전체 1개 중 1 - 1 < 1 > 10 / 페이지

Applied AI & Computer Vision Lab · Sungkyunkwan University

11

AutoML — 환경 설정

실습 환경 — 데이터 & 폴더

새 폴더 추가

사용 방식 ☒ 일반 ☐ 모델 ☐ 자동 마운트 ?

폴더 이름

위치

종류 ☒ 사용자

권한 ☒ 읽기 및 쓰기 ☐ 읽기 전용

초기화

취소

생성

데이터 & 폴더

이름으로 검색

C

+

☐ 이름 (경로 & 대체이름 ?)	사용 방식	호스트	종류	권한	생성됨
☐ SKKU	general	sp-bai-sc-g010:sas	사용자	R W	2026.07.0

☒ 이름 (경로 & 대체이름 ?)	사용 방식	호스트	종류	권한	
☒ SKKU					
☒ 경로 & 대체이름	general	sp-bai-sc-g010:sas	사용자	R W	
/home/work/SKKU					

AutoML — 환경 설정

실습 환경 — 검토 및 시작

새 세션 시작

최근 기록

세션 타입

수정

세션 타입: interactive

실행 환경

수정

프로젝트: Edu

이미지: Ngc Pytorch | 24.09 | x86_64 | PyTorch 2.5 | Python 3.10 | GPU: CUDA12.6

자원 할당

수정

자원 그룹: default

컨테이너당 리소스 할당: 18 Core 48 GiB (SHM: 1.00GiB) 0.30 fGPU

에이전트: auto 컨테이너 개수: 1

클러스터 모드 설정: 단일 노드

총 자원 할당량

18 Core 48 GiB (SHM: 1.00GiB) 0.30 fGPU

- 세션 타입: interactive
- 행 환경: NGC PyTorch 24.09 (PyTorch 2.5 · Python 3.10 · CUDA 12.6)
- 자원 할당: 18 Core · 48 GiB(SHM 1 GiB) · 0.30 fGPU, 단일 노드 / 컨테이너 1개
- 세 항목 일치 여부 검토 후 세션 시작

AutoML — 환경 설정

실습 환경

대시보드

스토리지

데이터

워크로드

세션

나의 실행 환경

메트릭

자원 요약

통계

세션을 만들어 보세요

세션 시작

나의 총자원 제한

CPU
18 Core

RAM
48 GiB

GPU
0.3 fGPU

사용 중

C

⚙

세션

C

세션 시작

전체 1

Interactive 1

Batch

Inference

업로드 세션

실행중

종료됨

세션 이름

검색

Q

세션 이름	상태	AI 가속기	CPU	메모리	경과 시간	실행 노드
150baIMX-session	RUNNING	0.30 fGPU	18	48 GiB	00:02:31	i-sc-g010

2

클릭

1

전체 1개 중 1 - 1

<

1

>

10 / 페이지

⚙

AutoML — 환경 설정

실습 환경

× 세션 정보

클릭

1S0baIMX-session

세션 ID	7363a1bb-1380-438c-8e13-3b70168adcc6		
상태	RUNNING		
실행 환경	Ngc Pytorch 24.09 x86_64 PyTorch 2.5 Python 3.10 GPU: CUDA12.6		
자원 할당	default 18 Core 48 GiB 0.30 fGPU		
예약 시간	2026년 7월 27일 오전 9:57 경과 시간: 00:07:52		
클러스터 모드	단일 (1)		
자원 사용량	CPU: 0.1% / 1800% RAM: 0.14 GiB / 48 GiB GPU (util): 0% GPU (mem): 0 GiB / 24 GiB I/O Read: 0 MB / Write: 0.26 MB		

커널

호스트명	상태	에이전트 ID	커널 ID	컨테이너 ID
main1	RUNNING	i-sc-g010	dd0963f2-c18c-42a8-8449-82af75c0c761	3beef358d6b918fa054249d73

전체 1개 중 1 - 1 < 1 > 10 / 페이지



앱: 1S0baIMX-session

Utilities

Console

Development

Jupyter Notebook

JupyterLab

Visual Studio Code

Machine Learning Tools

TensorBoard

사전 개방 포트

mlflow-ui

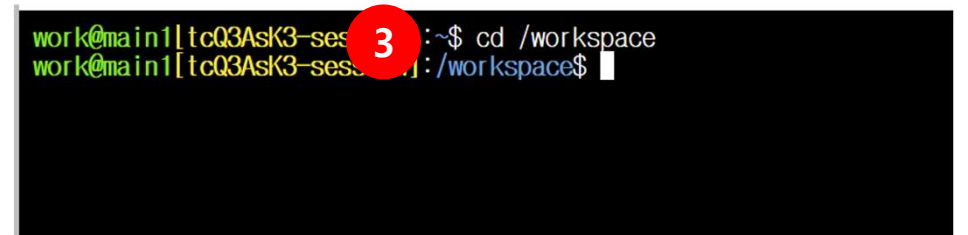
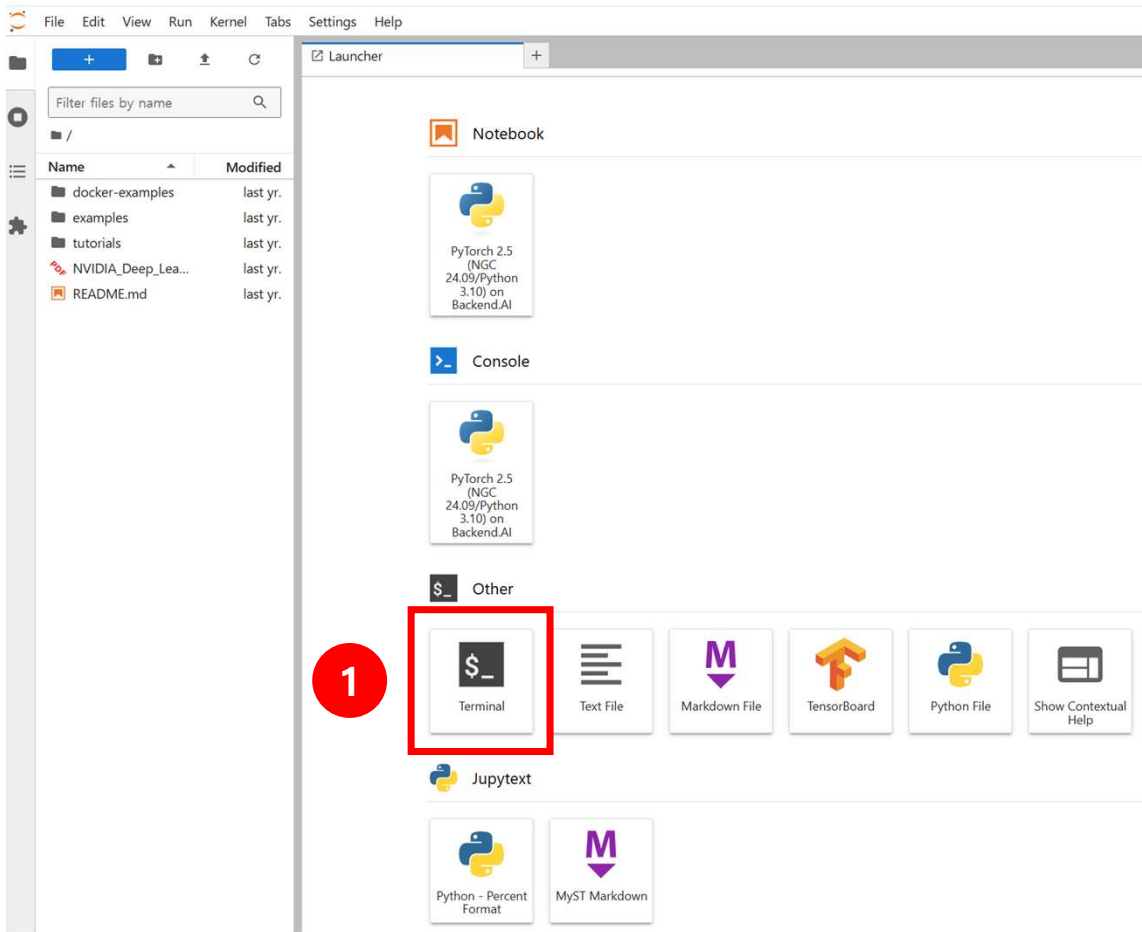
nniboard

☐ 앱을 외부에 공개

허용된 클라이언트 IP (선택으로 구분)

AutoML — 환경 설정

실습 환경



cd /workspace

AutoML — 환경 설정

실습 환경

```
work@main1[tcQ3Ask3-session]:~$ git clone https://github.com/SE93W0/SKKU_SUMMER_LECTURE.git
Cloning into 'SKKU_SUMMER_LECTURE'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
```

Name	Modified
/	
docker-examples	last year
examples	last year
SKKU	2 days ago
SKKU_SUMMER_LECTURE	24 seconds ago
tutorials	last year
NVIDIA_Deep_Learning_Co...	last year
README.md	last year

Name	Modified
Day1	3h ago
Day2	7h ago

Name	Modified
tabular	4h ago
TimeSerie	3h ago
install.ipynb	3h ago
SeoulBikeData.csv	20d ago

```
[*]: !pip install -U pip
      !pip install -U setuptools wheel
      !pip install fg-data-profiling
      !pip install -U ipywidgets
      !pip install msgpack
      !pip install autogluon
      !pip install -U accelerate transformers
      !pip install -U bitsandbytes
```

클릭 -> Shift + Enter

AutoML

개요

머신러닝의 민주화 — 전문성은 그대로, 장벽만 낮춘다

AutoML은 전문가를 대체하지 않고, 반복 작업을 대신 수행 — 연구자는 문제와 해석에 집중

연구자가 하는 일

- 데이터 수집 및 품질 확인
- 예측할 대상 설정
- 의미 있는 변수 판단
- 결과 해석·활용

AutoML이 하는 일

- 코드·환경 설정 (코드 몇 줄)
- 전처리·검증 반복 작업
- 여러 모델 학습·비교·튜닝
- 성능 리더보드 제공 및 최적 모델 저장

AutoML — 개요

실습 로드맵

PART 1 정형 데이터

조건에 따른 값 예측

EDA
데이터 살펴보기

전처리
숫자 변환·특징 생성

모델 학습
여러 모델·앙상블

해석
변수 중요도·예측

PART 2 시계열 데이터

시간 순서로 미래 예측

EDA
시간 흐름·계절성 확인

전처리
시계열 형식으로 변환

모델 학습
지역·전역·사전학습

해석
예측 구간·변수 중요도

AutoML

정형(Tabular) 데이터

정형 데이터란?

정형 데이터 — 행과 열로 정리된 표 형태의 데이터

열 = 변수(특성)

↓

기온(°C)	습도(%)	강수량(mm)	시간	계절	공휴일	대여량(대)
20	55	0	8	봄	아니오	1,120
28	70	5	14	여름	아니오	430
-5	40	0	18	겨울	아니오	560
15	60	0	3	가을	예	90

행 = 한 사례 →

정답 열 = 예측 대상

정형 데이터로 무엇을 할 수 있나

표에서 '정답 열'을 정하면, 나머지 열로 그 답을 예측

분류

(Classification)

“어떤 종류인가?”

정답이 범주일 때

예: 합격/불합격, 정상/불량, 우수/보통/미흡

회귀

(Regression)

“값이 얼마인가?”

정답이 숫자일 때

예: 주택 가격, 수확량, 미세먼지 농도

변수 중요도

(Feature Importance)

“무엇이 영향을 주는가?”

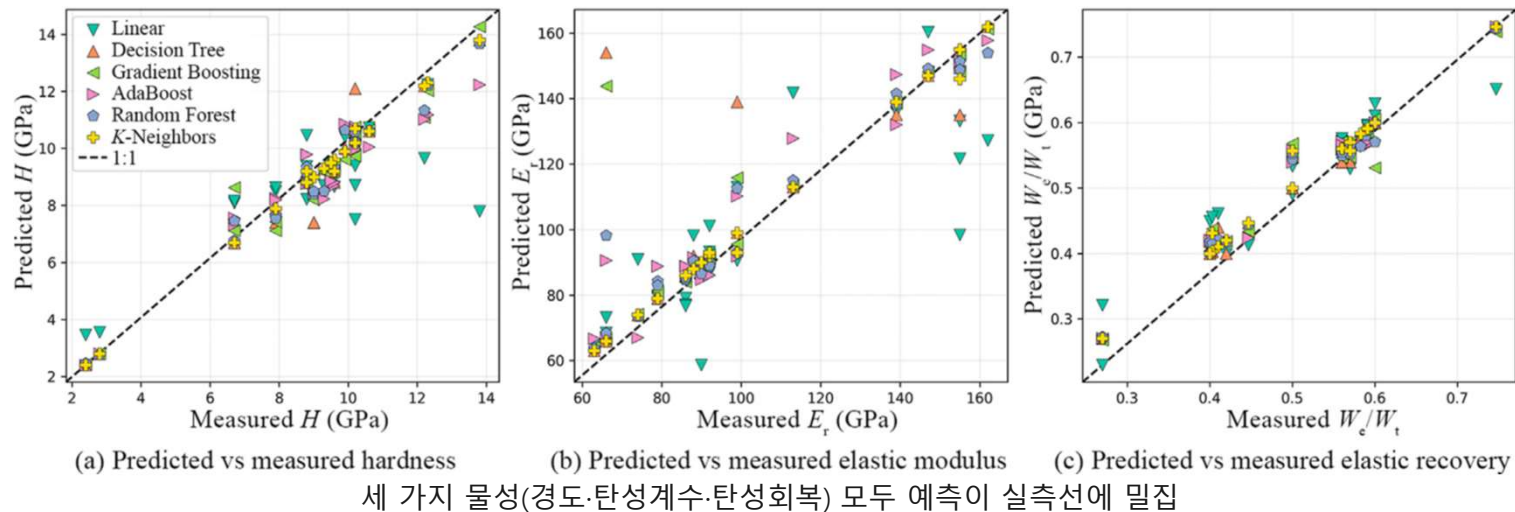
어떤 변수가 결과를 좌우하는지 파악

예: 성적에 영향을 준 요인, 불량을 유발한 공정 변수

AutoML — 정형 데이터

달 운석 광물 물성 예측

희귀한 달 운석은 손상 최소화가 필수 — 원소 조성은 비파괴로 측정 가능하나, 물성은 직접 측정이 까다로움
→ 비파괴로 얻는 원소 조성만으로 물성까지 예측할 수 있을까?



데이터

현미경으로 측정 원소 비율 표 (시료 1개 = 1줄)
시료A → 산소45%·마그네슘12%·규소20%·철8%

모델

분류 = TabPFN (트랜스포머)
물성 예측 = K-최근접이웃(K-NN) 등 6종 비교

결과

광물·광물군 분류 100% (운석 분류 92.3%)
물성 예측 $R^2 \approx 0.99$ (1에 가까울수록 정확)

Machine learning applications on lunar meteorite minerals: From classification to mechanical properties prediction, Peña-Asensio et al., International Journal of Mining Science and Technology (2024)

AutoML — 정형 데이터

합금 설계

금속 배합 조합이 사실상 무한 — 실험만으로 최적 조성 탐색은 비현실적
→ 데이터로 최적 조성을 찾고, 강한 이유까지 설명할 수 있을까?

역설계는 어떻게?

조성→물성은 계산되지만, 그 반대(물성→조성)는 어려움 → 후보를 넣어보며 찾아야 함

- ① **목표:** 원하는 물성을 기준으로 설정 (예: 더 단단하게)
- ② **빠른 예측:** 물성 계산은 시뮬레이션이라 느림 → AI가 대신 즉시 예측(대리 모델) → 수만 번 시도 가능
- ③ **반복 개선:** 더 나은 조성을 제안·평가·수정하며 목표에 수렴

데이터

합금 1개 = 한 줄(원소 5종 비율)
정답 열 = 물성 · 시뮬레이션으로 수만 건 생성

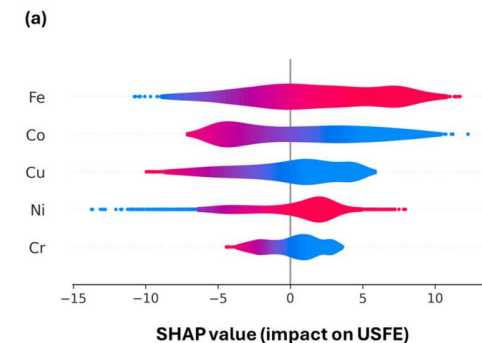
모델

물성 예측 + 최적 조합 거꾸로 탐색 + 이유 설명
모델: 앙상블 ML·1D CNN

결과

표준(등물) 합금보다 우수한 새 조합 발굴
→ 실제로 합성·검증 (FCC, 예측과 일치)

원소별 영향력 (SHAP) — 위로 갈수록 큼



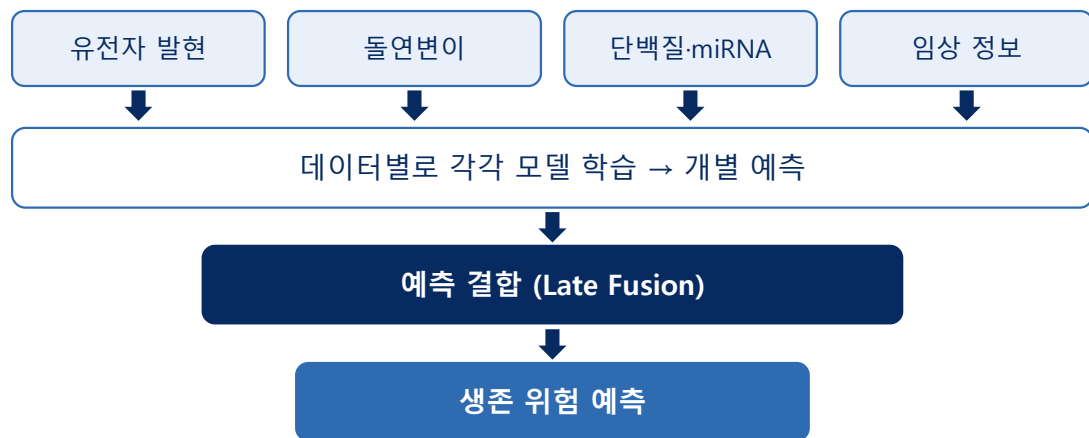
Fe-Co가 물성을 가장 크게 좌우, Cr은 미미

Experimentally validated inverse design of FeNiCrCoCu MPEAs and unlocking key insights with explainable AI, Wang et al., npj Computational Materials (2025)

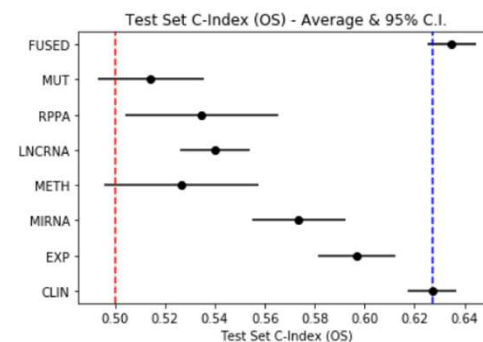
AutoML — 정형 데이터 암환자 생존 예측

검사 종류는 많지만 환자 수는 적음 — 데이터를 그냥 합치면 과적합으로 오히려 부정확
→ 적은 데이터로도 여러 검사를 잘 활용해 정확히 예측할 수 있을까?

데이터별로 각각 모델링 후 예측을 결합



융합 모델(FUSED)이 가장 정확



FUSED가 맨 위·가장 오른쪽 = 단일 데이터보다 정확

데이터

환자 1명당 여러 검사를 모은 표
유전자·돌연변이·단백질·임상 (4~7종)

모델

검사 종류별로 따로 모델 학습
예측 결합 — Late Fusion(앙상블)

결과

33개 암종 중 24개, 단일 대비 우수
범암종 C-index 0.785 (0.5=찍기, 1=완벽)

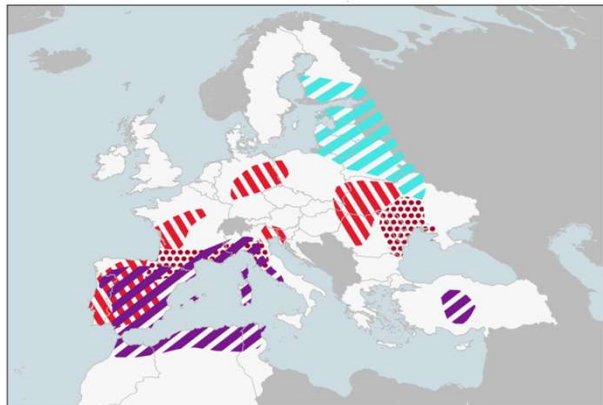
A machine learning approach for multimodal data fusion for survival prediction in cancer patients, Nikolaou et al., npj Precision Oncology (2025)

AutoML — 정형 데이터

기후 재해 탐지

기후 데이터는 폭증하지만 전문가 수작업 탐지는 느리고 주관적
→ AI로 빠르고 일관되게, 근거까지 제시하며 탐지할 수 있을까?

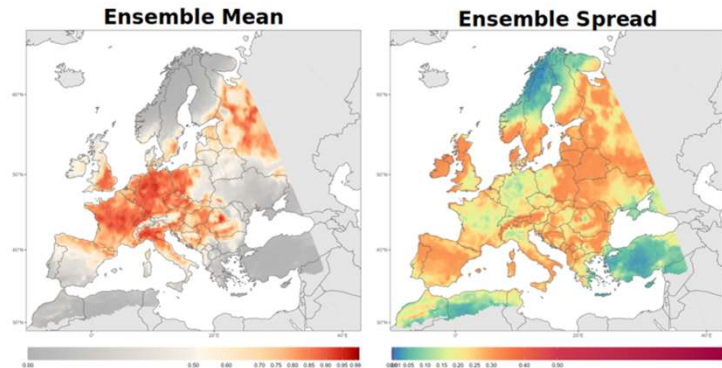
BEFORE — 전문가가 손으로 그린 위험 지도



Cold spell
Drought
Rain deficit
Heat wave



AFTER — 모델이 자동 생성 (위험 + 불확실성)



지역별 재해 위험 확률 + 불확실성 (폭염·가뭄 등 다중 재해)

데이터

지역·시점별 날씨 측정값 표
1줄 = 특정 지역·시점 (기온·강수 등)

모델

전문가 정답 지도로 학습 (XGBoost 앙상블)
다중 재해를 확률로 판정

결과

다중 재해 확률지도 + 불확실성 제공
SHAP로 판단 근거(주요 변수) 제시

Expert-driven explainable AI models can detect multiple climate hazards relevant for agriculture, Essenfelder et al., Communications Earth & Environment (2024)

AutoML

정형 데이터 전처리

전처리는 왜 필요한가?

머신러닝 모델이 이해하는 건 숫자

전처리가 하는 두 가지 일 — 숫자로 변환, 의미 있는 특징 만들기

모델의 언어(숫자)로 변환

글자·날짜·범주처럼 모델이 인식하지 못하는 값은
숫자로 바꿔야 학습 가능

예시

겨울 → 계절=4

의미 있는 특징 만들기

원시 값에 묻혀 있던 정보를 모델이 쓸 수 있는 특징(feature)으로 추출

예시

2017-12-01 → 요일·주말 여부·계절

AutoML — 전처리

전처리란 무엇인가?

BEFORE — 원본 (글자·빈칸이 섞인 표)

날짜	기온(°C)	계절	대여량
12/01	-5.2	Winter	254
12/01	(빈칸)	Winter	204
12/02	-6.0	Winter	173



AFTER — 전처리 후 (모두 숫자, 빈칸 없음)

월	기온(°C)	계절	대여량
12	-5.2	3	254
12	-5.6*	3	204
12	-6.0	3	173



Imputation

빈 값 채우기

비어 있는 칸을 평균·중앙값이나 비슷한 사례 값으로 채움

AutoGluon



Encoding

글자를 숫자로 바꾸기

ML이 못 읽는 글자(범주)를 숫자로 바꿈

예: 계절 봄·여름·가을·겨울 → 1·2·3·4

AutoGluon



Feature Engineering

특징 만들고 고르기

새 정보를 만들고, 예측에 중요한 항목만 선별

예: 날짜 → 월·요일, 시간대 추출

일부 AutoGluon · 핵심은 사람

AutoGluon의 자동 Feature Engineering

항목	예시 (before → after)
열 타입 자동 감지	Winter → 범주 / 2017-12-01 → 날짜 / 기온 -5.2 → 숫자
범주형 자동 인코딩	Spring, Summer, Autumn, Winter → 0, 1, 2, 3
날짜 자동 분해	2017-12-01 → 연 2017 · 월 12 · 일 1 · 요일 금
텍스트 자동 수치화	"서비스가 매우 좋았다" → 단어 빈도·글자수 등 숫자 특성
불필요한 열 자동 제거	값이 전부 '한국인'인 열, 다른 열과 완전 중복인 열 → 삭제

연구자가 수행해야 하는 Feature Engineering

해야 할 일	문제 상황 (예시)	조치
숫자로 코딩된 범주 지정	진료과코드 1.2.3 → 기계는 "3이 1의 3배"로 오해	<code>astype("category")</code>
숨은 결측 코드 처리	혈압 999 = '측정 안 됨' → 기계는 진짜 혈압으로 학습	<code>999 → NaN</code>
누수(leakage) 변수 제거	'퇴원일'로 재입원 예측 → 예측 시점엔 알 수 없는 값	해당 열 제거
구조에 맞는 데이터 분할	같은 환자가 train/test 양쪽에 → 성능 과대평가	환자·시간 단위 분할
도메인 파생 변수 생성	키·체중 따르는 약한 신호	$BMI = \text{체중} / \text{키}^2$ 생성

AutoGluon의 컬럼 타입 인식과 자동 처리

타입	예시 값	어떻게 알아내나	어떻게 처리하나
boolean	예 / 아니오	고유값이 딱 2개	0 / 1
numerical	1.3, -5.2	숫자형(int/float)	그대로 통과
categorical	Winter, Spring	문자열 + 반복되는 값	정수 매핑 (Winter→0 ...)
datetime	2017-12-01	날짜 변환에 성공	연·월·일·요일로 분해
text	Mary had a little lamb	문자열 + 대부분 고유 + 여러 단어	단어 빈도·글자수 등

AutoGluon의 자동 데이터 정리: 열 제거와 결측치 전략

쓸모없는 열 자동 제거

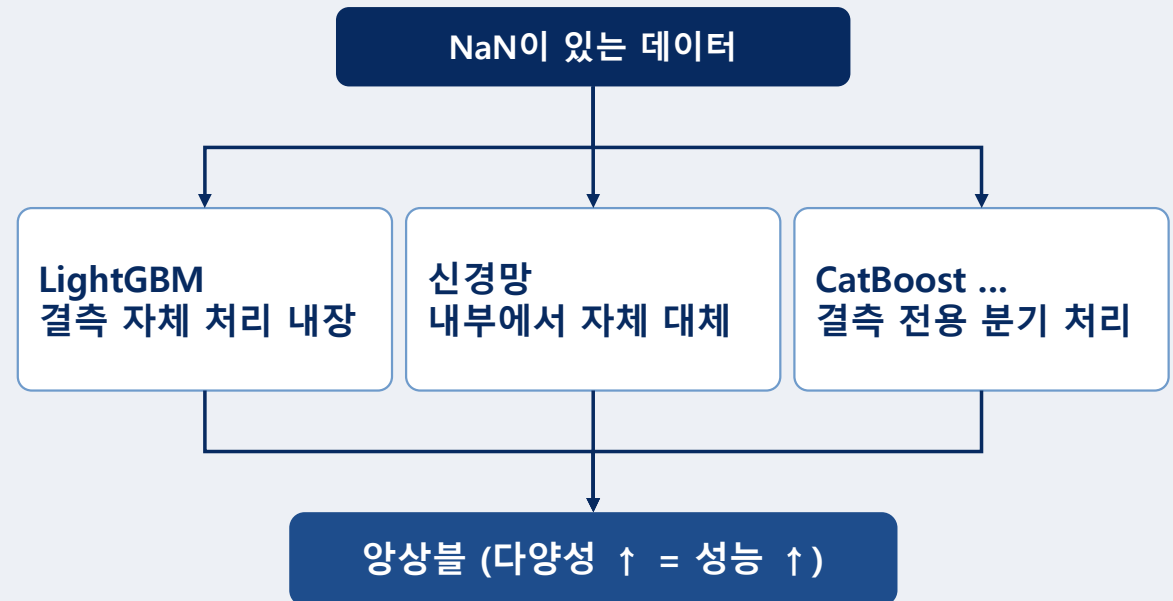
정보 없는 열은 알아서 삭제

나이	국가	나이_사본
34	한국	34
51	한국	51
27	한국	27

- **상수 열:** 국가가 전부 "한국" → 구분에 아무 도움 안 됨 → 삭제
- **중복 열:** 나이_사본 = 나이와 완전히 동일 → 같은 정보 두 번 → 삭제

결측치(NaN): 대체 없이 모델별 처리

채우지 않고 각 모델에 전달하는 전략



데이터 소개: 서울 공공자전거 대여 기록

서울시 공공자전거의 시간대별 대여량 + 그 시각의 날씨 기록

기간: 2017.12.01 ~ 2018.11.30 (1년) · 8,760행 (365일×24시간) · 14개 컬럼 · 결측 없음

날씨와 시간 정보로 시간당 대여량을 예측 → 회귀(regression) 문제

Date	대여량	Hour	기온(°C)	습도(%)	강수(mm)	Seasons	Holiday
01/12/2017	254	0	-5.2	37	0.0	Winter	No Holiday
01/12/2017	204	1	-5.5	38	0.0	Winter	No Holiday
01/12/2017	173	2	-6.0	39	0.0	Winter	No Holiday

AutoML — 전처리

데이터의 컬럼 구성과 타입 분류

컬럼	실제 값 예시	타입
Date	01/12/2017	datetime
Temperature(°C)	-17.8 ~ 39.4	numerical
Humidity(%)	0 ~ 98	numerical
Wind speed (m/s)	0 ~ 7.4	numerical
Visibility (10m)	27 ~ 2000	numerical
Dew point temperature(°C)	-30.6 ~ 27.2	numerical
Solar Radiation (MJ/m2)	0 ~ 3.52	numerical
Rainfall(mm) / Snowfall (cm)	0 ~ 35 / 0 ~ 8.8	numerical
Hour	0 ~ 23	numerical
Seasons	Winter·Spring·Summer·Autumn (4개)	categorical
Holiday	Holiday · No Holiday (2개)	boolean
Functioning Day	Yes · No (2개)	boolean
Rented Bike Count	0 ~ 3,556	예측 대상 (label)

탐색적 데이터 분석(EDA)

모델 적용 전, 전체 분포·품질을 먼저 파악하는 단계
전처리 대상은 이 단계의 결과에 따라 결정

EDA에서 확인하는 것

- **분포:** 값이 몰린 구간, 치우침 여부
- **결측·이상치:** 빈 값, 999.-1처럼 숫자로 위장한 결측, 범위를 벗어난 값
- **중복·불필요한 열:** 값이 하나뿐인 열, 다른 열과 동일한 열
- **변수 간 관계:** 예측 대상과 함께 움직이는 변수

전처리와의 관계

- **위치:** 데이터 수집 다음, 전처리 이전 단계
- **처리 대상:** EDA에서 드러난 결측·위장 결측·상수 열·분포 특성
- **연계:** 전처리 작업은 EDA 결과에 근거

자동 EDA 도구 — FG Data Profiling

```
from data_profiling import ProfileReport
ProfileReport(df, title="따릉이 EDA").to_file("eda.html")
```

섹션	확인 내용
Overview	행·열 수, 결측·중복 총량, 데이터 품질 경고(alert)
Variables	변수별 분포·통계량 (평균·최소·최대·고유값 수)
Correlations	변수 간 상관관계 (수치·범주)
Missing values	결측 위치·비율
Duplicate rows	중복 행
Sample	데이터 앞·뒤 일부 미리보기

리포트 읽기 — 발견에서 전처리로

리포트의 각 섹션에서 확인한 사항 → 전처리 단계의 처리 대상으로 연결

리포트에서 확인	따릉이 데이터의 발견	전처리 조치
Variables — Hour 분포	오전·오후 두 번 피크 (이봉 분포)	Hour를 범주형으로 지정
Overview — Alerts	상수 열·중복 열 경고	해당 열 제거
Variables — 최소·최대값	999.-1 등 숫자로 위장한 결측 점검	발견 시 NaN으로 변환
Missing values	결측 0건 (본 데이터 기준)	별도 대체 불필요
Correlations	대여량과 함께 움직이는 변수 (기온 등)	주요 변수 파악

AutoML — 전처리

데이터 불러오기와 원본 확인

```
import pandas as pd
from autogluon.features.generators import AutoMLPipelineFeatureGenerator

df = pd.read_csv("SeoulBikeData.csv", encoding="latin-1")
df["Date"] = pd.to_datetime(df["Date"], format="%d/%m/%Y")
df.head()
```

	Date	Rented Bike Count	Hour	Temperature(°C)	Seasons	Holiday	Functioning Day
0	2017-12-01	254	0	-5.2	Winter	No Holiday	Yes
1	2017-12-01	204	1	-5.5	Winter	No Holiday	Yes
2	2017-12-01	173	2	-6.0	Winter	No Holiday	Yes
3	2017-12-01	107	3	-6.2	Winter	No Holiday	Yes
4	2017-12-01	78	4	-6.0	Winter	No Holiday	Yes

▲ 모델이 못 읽는 문자열(범주형) 열 — Winter · No Holiday · Yes

자동 Feature Engineering: fit_transform

```
X = df.drop(columns=["Rented Bike Count"]) # 예측 대상 제외
generator = AutoMLPipelineFeatureGenerator()
X_transformed = generator.fit_transform(X)
```

변환 전

Date	Seasons	Holiday
2017-12-01	Winter	No Holiday
2017-12-01	Winter	No Holiday
2017-12-01	Winter	No Holiday

fit_transform



변환 후

year	month	day	dayofweek	Seasons	Holiday
2017	12	1	4	3	1
2017	12	1	4	3	1
2017	12	1	4	3	1

결측치(NaN) 처리 방식

```
df_missing = X.copy()
df_missing.loc[0, "Temperature(°C)"] = np.nan # 첫 행 기온을 일부러 비움
out = AutoMLPipelineFeatureGenerator().fit_transform(df_missing)
print("첫 번째 행의 기온:", out["Temperature(°C)"].iloc[0])
```

변환 전

Temperature(°C) = **NaN**

fit_transform



변환 후

Temperature(°C) = **nan**

AutoGluon이 하는 일

- 결측치를 임의로 채우지 않고, 각 모델에 **그대로 전달**
- 모델마다 고유한 방식으로 처리
→ 모델 다양성 ↑, 앙상블 성능 ↑

결론: 연구자가 결측치를 미리 채울 필요 없음

연구자가 해야 하는 일

- 999 · -1처럼 숫자로 위장한 결측은 **AutoGluon이 알아채지 못함** → 직접 변환
- `df["열"] = df["열"].replace(999, np.nan)`

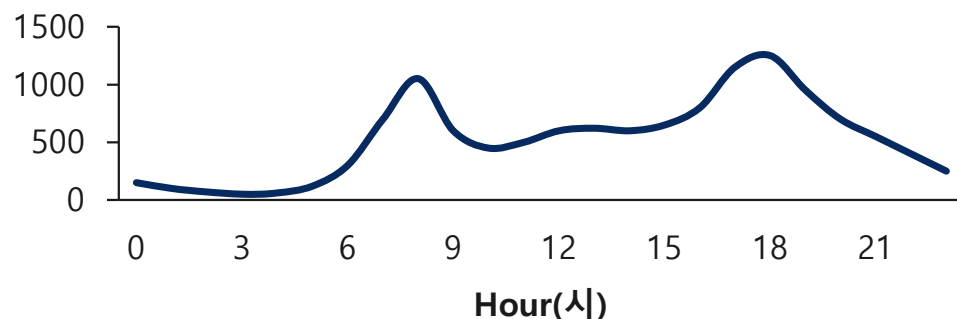
AutoML — 전처리

사용자 개입: Hour를 범주형으로 지정

```
# 기본 처리 - Hour는 숫자로 취급됨
out_default = AutoMLPipelineFeatureGenerator().fit_transform(X.copy())
print("기본 처리 시 Hour 타입:", out_default["Hour"].dtype)

# 사용자 개입 - Hour를 범주형으로 지정
df_hour = X.copy()
df_hour["Hour"] = df_hour["Hour"].astype("category")
out_hour = AutoMLPipelineFeatureGenerator().fit_transform(df_hour)
print("astype 적용 후 Hour 타입:", out_hour["Hour"].dtype)
```

시간대별 평균 대여량 — 오전·오후 두 번 피크



Hour를 숫자로 두면

‘시간이 커질수록 늘거나 준다’는 하나의 규칙만 찾음
→ 실제론 오전·오후 두 번 피크 → 곡선을 뭉갸

Hour를 범주로 두면

0시 ... 23시를 각각 따로 학습
→ ‘8시는 높음, 4시는 낮음’을 시간대별로 파악

실습 ① — 데이터 준비와 학습 전 확인

따라 하기 · 따름이

강의에서 본 불러오기 → EDA 그대로 실행

흐름 익히기

1 · 불러오기

구분자 · 결측 표기 확인

2 · EDA

ProfileReport(df) 로 분포 · 결측 · 상관 확인

3 · 학습 전 확인

위장결측 · 누수 변수 · 숫자로 위장한 범주

AutoML — 전처리

실습 데이터셋

Pump it Up — 탄자니아 급수시설 상태 예측 · 3분류 · 59,400행

급수펌프의 설비·위치·관리 정보로 작동 상태를 예측하는 정부·시민 결합 데이터

Absenteeism — 직장 결근 시간 예측 · 회귀 · 740행

브라질 택배회사 직원 36명의 교통비·통근거리·건강 정보로 결근 시간을 예측

Diabetes 130 — 당뇨 환자 재입원 예측 · 분류 · 101,766행

미국 130개 병원의 당뇨 입원 기록으로 30일 내 재입원을 예측하는 전자의무기록 10만 건

Heart Failure — 심부전 환자 사망 예측 · 이진분류 · 299행

심부전 환자 299명의 혈액검사값으로 사망을 예측하는 소규모 의료 데이터

AutoML — 전처리

실습 데이터셋

Myocardial Infarction — 심근경색 합병증 예측 · 분류 · 1,700행

병력·검사·심전도 124개 변수로 합병증을 예측하는 결측·불균형·누수 고난도 데이터

AI4I 2020 — 설비 고장 예측 · 이진분류 · 10,000행

밀링 머신의 온도·회전속도·토크·마모로 고장을 예측하는 예지보전 표준 데이터

Concrete — 콘크리트 압축강도 예측 · 회귀 · 1,030행

시멘트·물·골재 배합비와 양생기간으로 강도를 예측하는 재료공학 데이터

Superconductivity — 초전도체 임계온도 예측 · 회귀 · 21,263행

원소 조성에서 계산한 물성으로 임계온도를 예측해 신소재를 실험 전 선별

AutoML — 전처리

실습 데이터셋

Crop Recommendation — 작물 추천 · 다중분류 · 3,867행

에티오피아 농지의 토양 성분과 계절 기후로 12종 중 적합 작물을 추천

Forest Fires — 산불 피해 면적 예측 · 회귀 · 517행

포르투갈 공원의 기상 조건으로 산불 소실 면적을 예측하는 저신호 난제 사례

* 각 데이터 상세는 *data_*.txt* 참조

AutoML

정형 데이터 모델 학습

다양한 모델의 학습과 앙상블

최적의 모델은 사전에 알 수 없다

→ AutoGluon은 다양한 모델을 학습한 뒤 성능을 비교하여 선택

트리 계열

LightGBM · XGBoost · CatBoost · RandomForest · ExtraTrees

신경망

NeuralNet

거리 기반

KNN

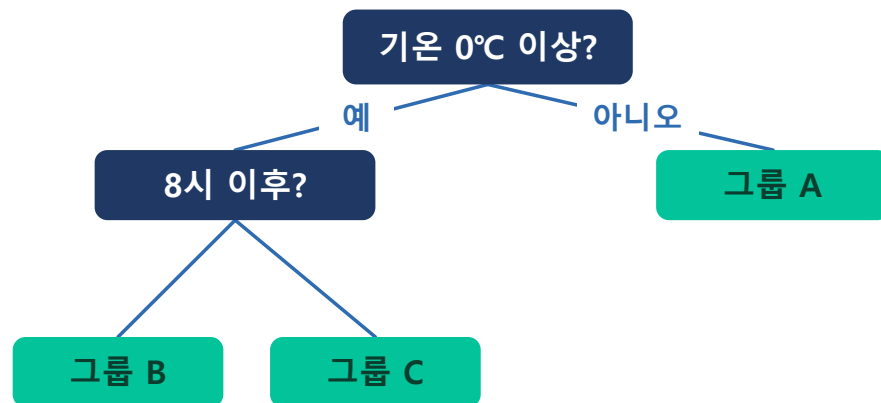
파운데이션

TabPFN

모델 — 트리 계열과 신경망

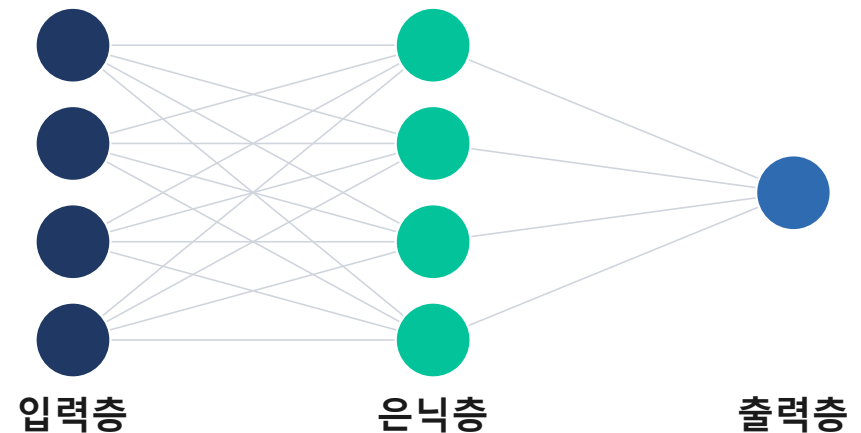
트리 계열

- 데이터를 조건에 따라 반복적으로 분할하여 예측하는 모델
- 각 분기에서 변수 기준으로 데이터를 나누고, 최종 구간의 값으로 예측
- 정형 데이터에서 가장 널리 쓰이는 주력



신경망

- 입력 변수를 여러 층으로 결합하여 복잡한 비선형 관계를 학습하는 모델
- 표 데이터에 맞게 설계된 형태
- 트리 모델과 결정 방식이 달라, 함께 사용 시 상호 보완



모델 — 거리 기반과 파운데이션 모델

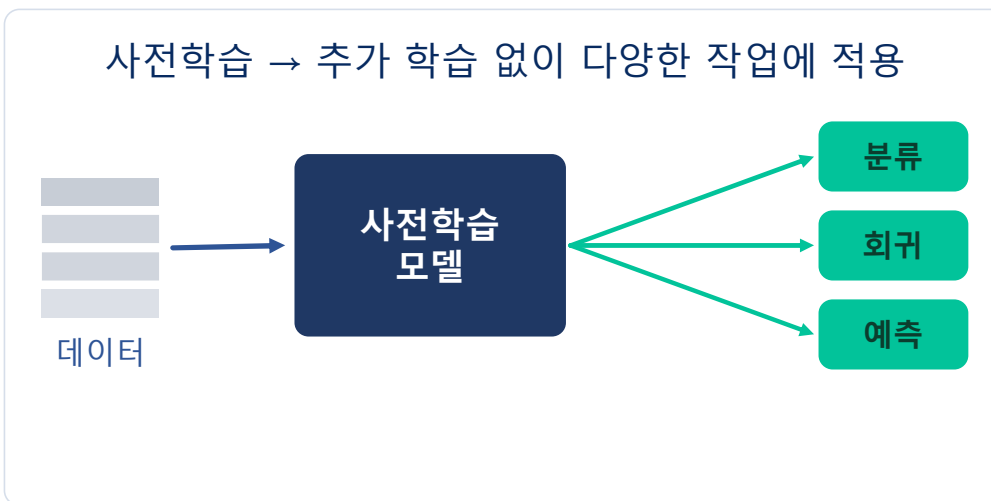
거리 기반

- 가장 가까운 기존 데이터로 새 값을 예측
- 변수 공간의 거리로 유사성을 판단
- 원리가 단순하고 직관적



파운데이션 모델

- 다른 모델: 내 데이터로 처음부터 학습
- 파운데이션: 이미 학습된 모델을 그대로 활용
- 소량의 데이터에서도 작동 (사전학습 지식 활용)



여러 모델을 안정적으로 — 배깅

왜

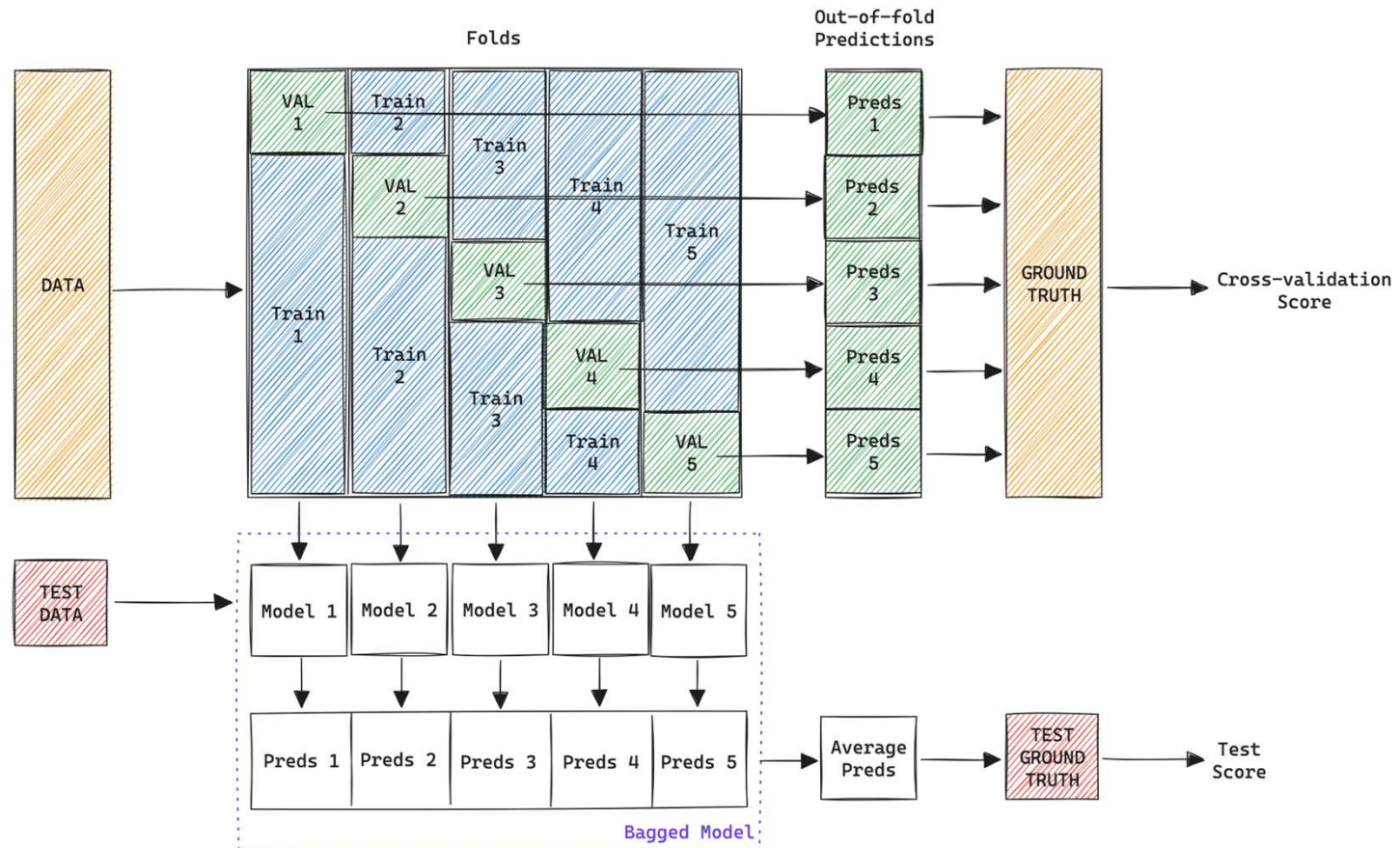
- 같은 모델도 데이터를 나눠 여러 번 학습하면 안정적
- 과적합 ↓ · 예측이 흔들리지 않음

어떻게 (교차검증과 결합)

- 데이터를 K개 조각으로 나눔
- 각 조각을 뺀 나머지로 학습 → K개 모델
(같은 알고리즘의 복사본 · 학습 데이터만 다름)
- 뺀 조각으로 검증 → 모든 데이터를 활용

결과

- 예측 = K개 복사본의 평균
- OOF 예측 생성 → 스택킹의 재료



모델을 쌓아 올리기 — 스택킹

서로 다른 모델을 쌓음

- 배깅 = 같은 모델 K개
- 스택킹 = 서로 다른 N종 (*LightGBM · XGBoost · 신경망 ...*)

다층 구조 (케이크처럼)

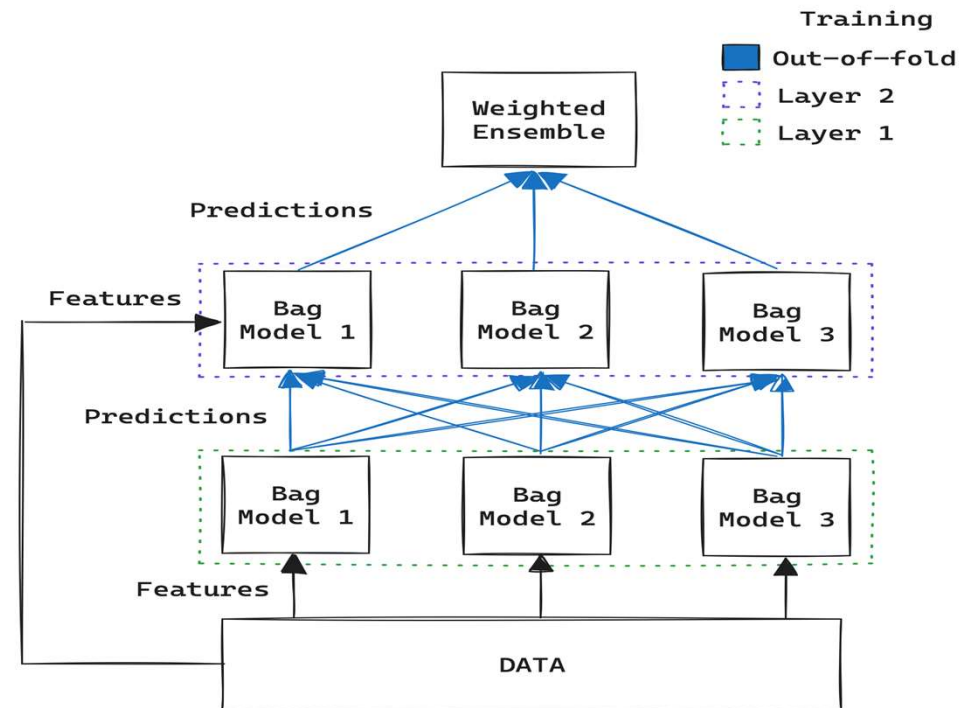
- 각 층: 여러 종류의 배깅 모델
- 다음 층 = 앞 층 예측 + 원본 데이터
- 첫 층은 원본 데이터만

최종 층 (가중 앙상블)

- 앞 예측들을 조합하는 "슈퍼 모델"
- 각 의견에 가중치 → 최종 결정 (*의견을 종합하는 리더*)

핵심

- OOF 예측 사용 → 누수 방지
- 자기가 학습한 데이터는 예측에 안 씀

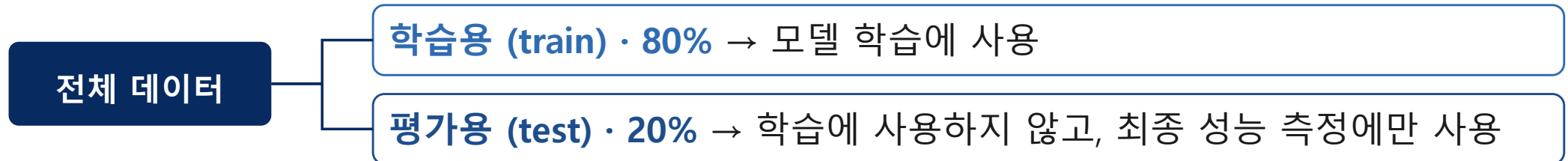


데이터 분할의 목적

분할이 필요한 이유

- 학습에 사용한 데이터로 성능을 측정하면, 모델이 이미 접한 사례를 다시 평가하는 것이므로 성능이 실제보다 높게 측정됨
- 학습에 사용하지 않은 데이터로 평가해야 일반화 성능을 확인할 수 있음

분할 구조



- 일반적인 비율: 80/20 또는 70/30
- 학습용 ↑ = 모델이 더 잘 배움 / 평가용 ↑ = 성능 측정이 더 안정적
- 데이터가 클수록 평가용 비율을 줄여도 됨 (90/10 등)
- 기준은 비율보다 평가용의 절대 개수 — 너무 적으면 측정이 불안정

분할의 편향: 소규모 데이터에서의 주의점

문제 상황

예: 제품 200개 중 불량품이 20개인 데이터

- 무작위 분할 시, 우연에 의해 불량품이 한쪽에 집중될 수 있음
- 평가용에 불량품이 2~3개만 포함되면 → 평가 결과가 불안정
- 학습용에서 불량품이 부족하면 → 모델이 불량 패턴을 학습하지 못함

데이터 규모에 따른 차이

대규모 데이터

무작위 분할로도 양쪽 분포가 유사해짐

소규모 데이터

우연에 의한 쓸림 위험이 커짐

해결 방안: 층화 추출 (Stratified Sampling)

주요 그룹의 비율이 학습용과 평가용에서 동일하도록 분할

예: 불량률 10% → 학습용·평가용 모두 10% 유지

※ AutoGluon은 내부 검증 분할에 층화 추출을 자동 적용 (분류 문제)

※ 연구자가 수행하는 평가용 분할은 연구자가 직접 확인해야 함

실습: 학습용과 평가용 데이터 분할

학습용 · 평가용 데이터 분할 (80 : 20)

```
train_data = df.sample(frac=0.8, random_state=42) # 80%를 무작위 추출 → 학습용
test_data  = df.drop(train_data.index)           # 나머지 20% → 평가용

print("학습용:", train_data.shape)
print("평가용:", test_data.shape)
```

출력 ▶ 학습용: (7008, 14) 평가용: (1752, 14)

분할 후 편향 점검 — 계절(Seasons) 분포가 양쪽에 고르게 들어갔는지 확인

```
ratio = lambda d: d["Seasons"].value_counts(
    normalize=True).round(3) * 100

pd.DataFrame({
    "학습용 비율(%)": ratio(train_data),
    "평가용 비율(%)": ratio(test_data),})
```

Seasons	학습용 비율(%)	평가용 비율(%)
Autumn	24.8	25.6
Spring	25.3	24.9
Summer	25.3	24.9
Winter	24.7	24.5

AutoML — 모델 학습

실습: fit() 한 줄로 학습시키기

fit() 한 줄로 학습 실행

```
from autogluon.tabular import TabularPredictor
predictor = TabularPredictor(label="Rented Bike Count").fit(train_data)
```

학습 로그

AutoGluon infers your prediction problem is: **'regression'**
Fitting model: LightGBM ... -150.9 = Validation score
Fitting model: RandomForestMSE ... -163.3 = Validation score
Fitting model: CatBoost ... -152.1 = Validation score
Fitting model: NeuralNetTorch ... -145.7 = Validation score
Fitting model: WeightedEnsemble_L2 **-138.9** = Validation score
AutoGluon training complete
Best model: **WeightedEnsemble_L2**

로그에서 확인할 점

- **문제 유형 자동 판별** — 'regression': 대여량이 숫자여서 회귀로 판단
- **여러 모델을 차례로 학습** — 각 모델 옆에 검증 점수(Validation score) 표시
- **최종 모델 자동 선택** — Best model: 가장 우수한 것(대개 양상블)을 자동 지정

실습: 성능 확인 leaderboard()

leaderboard() — 모든 모델 성능 순위표

```
predictor.leaderboard(test_data)
```

출력 — leaderboard (평가용 기준 순위)

model	score_test	score_val	fit_time
WeightedEnsemble_L2	-140.8	-136.2	117.1
NeuralNetTorch	-151.3	-148.5	97.1
ExtraTreesMSE	-162.4	-157.3	4.9
RandomForestMSE	-163.3	-159.4	15.2

- 맨 위 행이 최우수 — WeightedEnsemble(앙상블)이 1위
- **score_test** = 평가용(새 데이터) 성능 · **score_val** = 학습 중 검증 성능
- 음수 점수는 정상 — 부호를 뒤집어 "높을수록 좋음"으로 통일 (-140.8 > -151.3)
- **fit_time** = 학습에 걸린 시간(초)

연구자가 조절하는 학습 파라미터

연구자가 조절하는 파라미터

옵션	역할	예
<code>presets</code>	성능 ↔ 속도 균형	<code>best_quality</code> / <code>good_quality</code> / <code>medium_quality</code>
<code>time_limit</code>	학습 최대 시간(초)	600 (10분)
<code>eval_metric</code>	성능 판단 기준	회귀 RMSE · MAE / 분류 accuracy · F1

코드 예시

```

predictor = TabularPredictor(
    label="Rented Bike Count",
    eval_metric="rmse" # 평가 기준 지정
).fit(
    train_data,
    presets="best_quality", # 정확도 우선
    time_limit=600) # 10분 제한

```

Model	score_test	score_val	fit_time
WeightedEnsemble_L3	-117.00	-123.74	2286.07
CatBoost_r9_BAG_L2	-117.32	-125.80	1832.64
ExtraTrees_r42_BAG_L2	-117.34	-126.32	1798.33
ExtraTreesMSE_BAG_L2	-117.53	-126.31	1798.37

표 데이터용 파운데이션 모델

AutoGluon은 대량의 표 데이터로 사전학습된 파운데이션 모델 내장
추가 학습 없이 예측 가능, 특히 소규모 데이터에 강력

포함된 파운데이션 모델

모델	특징	용도
TabPFNv2	소규모 데이터에 특화, 사전학습됨	분류 · 회귀
Mitra	AutoGluon 자체 개발 파운데이션 모델	분류 · 회귀
TabICL	대규모 표 데이터에 확장 가능	분류 전용

코드 예시

```
predictor_fm = TabularPredictor(  
    label="Rented Bike Count"  
).fit(  
    train_data,  
    hyperparameters={  
        "TABPFNv2": {},  
        "MITRA": {},  
    })
```

모델	score_test	score_val	Fit_time
TabPFN	-119.87	-110.66	1.39
Mitra	-135.16	-136.20	118.08

변수 중요도 측정 원리: 순열 중요도(Permutation Importance)

“이 변수가 없으면 예측이 얼마나 나빠질까?”를 직접 측정

원본 데이터

기온	습도	대여
20	60	340
25	55	480
18	70	210

오차 = 141

‘기온’을 뒤섞음

기온	습도	대여
25	60	340
18	55	480
20	70	210

오차 = 571

성능 하락폭 = $571 - 141 = 430 \rightarrow$ ‘기온’의 중요도

하락폭이 클수록

중요한 변수 — 없으면 예측이 크게 망가짐

하락폭 ≈ 0

예측에 거의 기여하지 않음

음수(-)이면

오히려 해로운 변수 — 제거 시 성능 향상 가능

측정 3단계

1. 정상 예측 — 원본 데이터로 예측해 오차 측정
2. 한 변수 뒤섞기 — ‘기온’ 값만 행끼리 무작위로 섞어 정답과의 관계를 끊고 다시 예측 $\rightarrow$ 오차 측정
3. 하락폭 = 중요도 — $571 - 141 = 430$ 이 곧 ‘기온’의 중요도

변수 중요도 확인 — feature_importance()

실행 코드

```
predictor.feature_importance(test_data)
```

출력 결과 (importance 내림차순, 상위 5개 변수)

feature	importance	stddev	p_value	의미
Hour	397.97	8.31	2.3e-08	시간대가 대여량을 가장 크게 좌우
Temperature (°C)	235.49	6.55	7.2e-08	기온이 두 번째 주요 요인
Date	175.03	5.67	1.3e-07	날짜(계절·추세) 영향 반영
Functioning Day	167.28	12.12	3.3e-06	운영일 여부가 수요를 좌우
Solar Radiation (MJ/m ²)	121.78	4.21	1.7e-07	일사량(날씨) 영향

importance: 클수록 중요 **stddev · p_value:** 그 값의 신뢰도 (p_value가 작을수록 확실히 유용한 변수)

학습한 모델 사용하기

코드

```
pred = predictor.predict(new_data) # 새 데이터를 예측

loaded = TabularPredictor.load("seoul_bike_model") # 새 세션 . 다른 PC
loaded.predict(new_data) # fit()이 이미 저장했음
```

BEFORE — 정답이 없는 새 데이터

Date	Hour	기온	Seasons	대여량
2018-12-01	8	-3.0	Winter	?
2018-12-01	9	-1.5	Winter	?



AFTER — 정답 열이 채워짐

Date	Hour	기온	Seasons	대여량
2018-12-01	8	-3.0	Winter	412.6
2018-12-01	9	-1.5	Winter	388.1

실습 ② — 학습과 해석

따라 하기 · 따름이

강의에서 본 학습 → 해석 과정 그대로 실행

앞의 실습 데이터셋 중 하나 선택 (실습①의 준비 과정 적용)

4 · 학습

`TabularPredictor(label=...).fit(train) · leaderboard`

5 · 해석

`feature_importance` 로 변수 중요도 확인

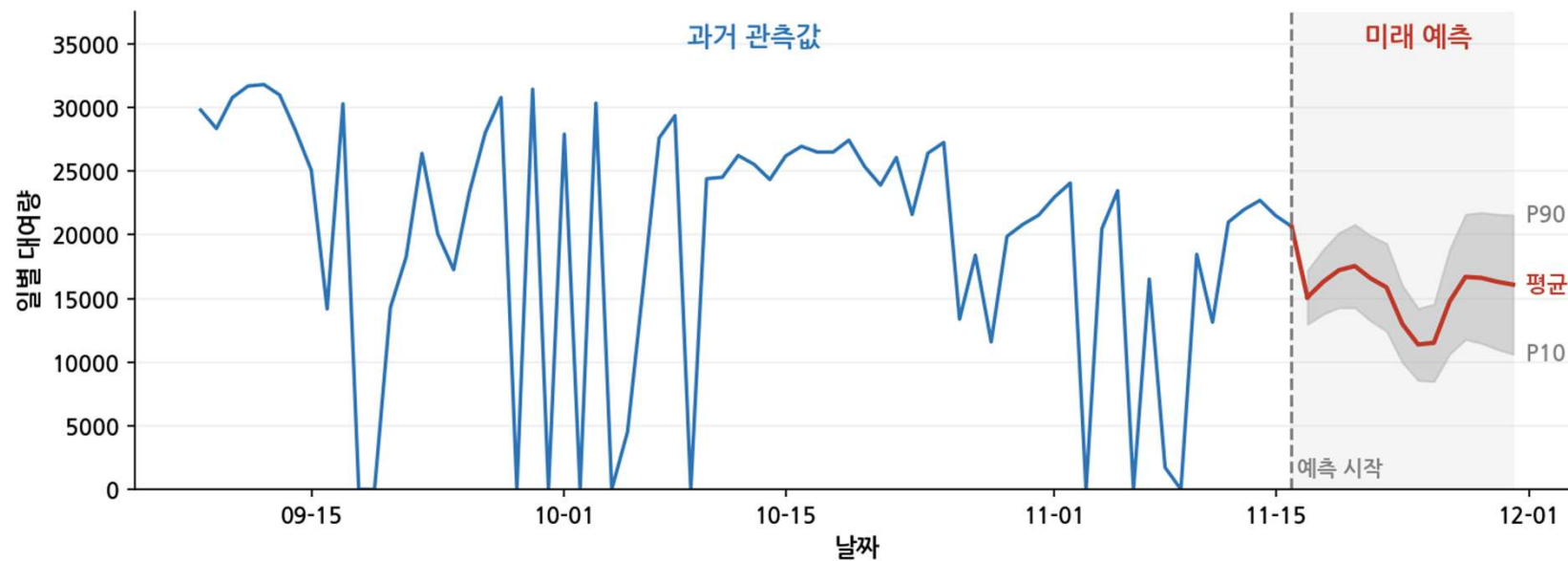
• 결과가 맞는지 판단

AutoML

시계열(Time-series) 데이터

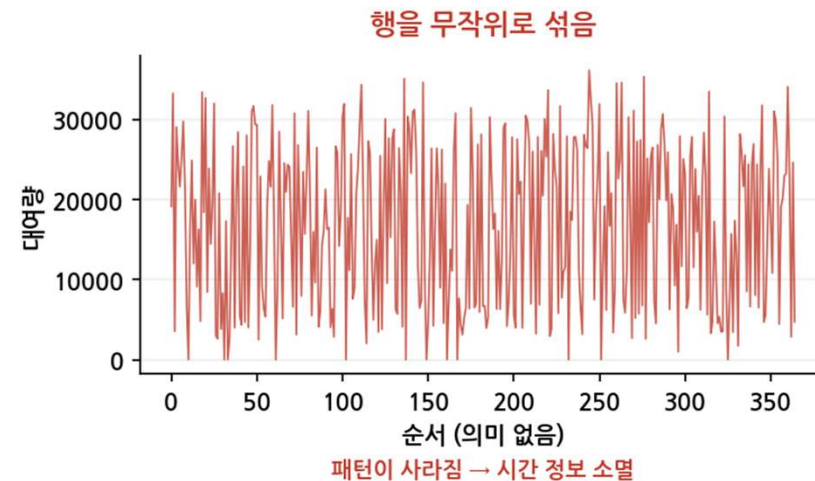
시계열 데이터: 시간 순서를 가진 데이터

시계열 데이터 = 시간 순서에 따라 기록된 데이터



정형 데이터와 시계열 데이터의 차이

핵심 차이는 '순서' — 행을 섞어도 되는가



항목	정형 데이터	시계열 데이터
순서	무관 — 섞어도 됨	핵심 — 섞으면 안 됨
분할	무작위 분할	시간 순 (과거→학습, 미래→평가)
예측 목표	조건에 따른 값	미래의 값

시계열 데이터로 무엇을 할 수 있나

시간의 흐름 속에서 미래를 예측하고, 패턴을 파악

예측

Forecasting

“이 흐름은 어디로 갈까?”

과거의 추세와 주기를 학습해 다음 시점을 추정

추세·계절성 분석

Decomposition

“어떤 흐름과 주기가 있나?”

장기적 추세와 반복되는 계절성을 파악

추세: 장기적 증감 방향 · 계절성: 주기적 반복

이상 탐지

Anomaly

“평소와 다른 때는 언제인가?”

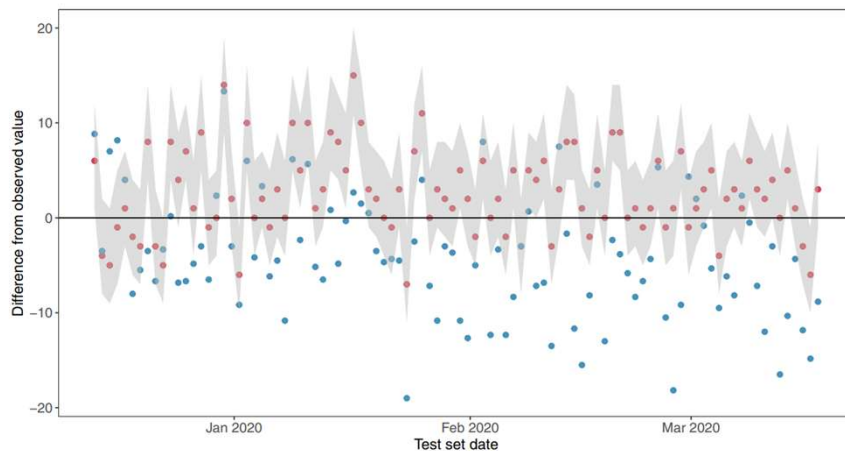
정상 패턴에서 벗어난 지점을 감지

AutoML — 시계열 데이터

응급실 입원 수요 실시간 예측

응급실 과밀은 대기·치료 지연, 환자 안전 위협을 초래
→ 실시간 진료 기록으로 입원 수를 미리 예측해 병상·의료진을 배치할 수 있을까?

실시간 전자의무기록으로 4.8시간 뒤 입원 수를 확률분포로 예측



0.82–0.90

AUROC · 입원 수요 예측 성능

4.0명

평균 오차(MAE) · 기존 관행 6.5명

데이터

- EHR 응급실 방문 21만여 건
- 2019~2021 · 입원율 20.7%
- 시간 순 분할(과거→최근)

모델

- XGBoost로 환자별 입원 확률
- 확률 합산 → 총 입원 수 예측
- RF·로지스틱회귀 대비 최우수

결과

- AUROC 0.82~0.90
- MAE 4.0명 (기존 6.5명)
- 실병원 실시간 시스템 운영

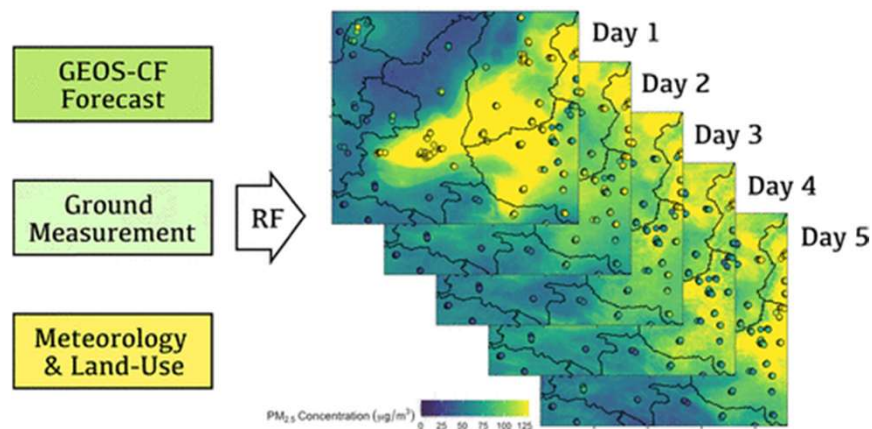
Machine learning for real-time aggregated prediction of hospital admission for emergency patients, King et al., npj Digital Medicine 5, 104 (2022)

고해상도 대기오염(PM2.5) 예측

기존 대기질 예보는 물리·화학 시뮬레이션에 의존 — 정확하나 연산이 무겁고 편향이 큼
→ 머신러닝으로 5일 앞 오염도를 촘촘한 지도로 더 가볍게 예측할 수 있을까?

향후 5일 PM2.5를 1km 해상도로 예측 (중국 중부 펜웨이 평원)

Spatiotemporal Five-Day PM2.5 Concentration Forecast



0.76

1일 뒤 R^2 · 2일 뒤 0.64

1km

고해상도 · 5일 앞 예측

데이터

- 지상 관측 + NASA 공개 위성·예보 데이터
- 특별한 장비 없이, 공개 데이터로 구성

모델

- 랜덤포레스트 앙상블
- GEOS-CF 예보를 학습
- 트리 기반 편향 보정

결과

- 1일 $R^2=0.76$, 2일 0.64
- 5일까지 예측 산출
- 물리모델 편향 보정·경량

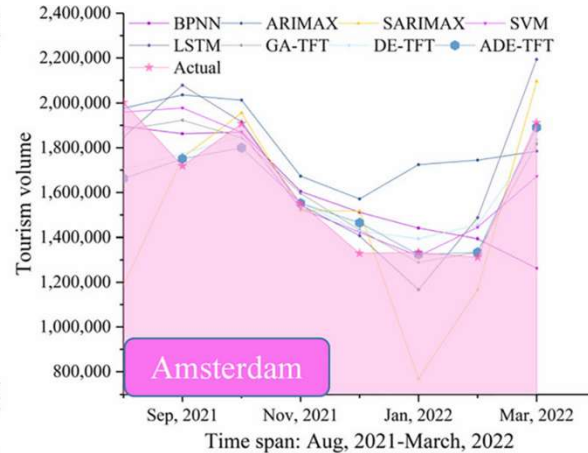
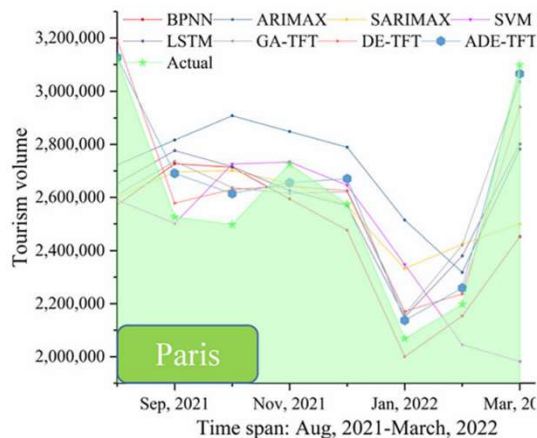
Combining Machine Learning and Numerical Simulation for High-Resolution PM2.5 Concentration Forecast, Bi et al., Environmental Science & Technology 56, 1544–1556 (2022)

AutoML — 시계열 데이터

관광 수요 예측 (코로나19 시기)

코로나19로 관광 수요가 급변 — 과거 데이터만으로는 예측이 어려움
→ 확진자·검색 트렌드를 결합해 급변하는 관광 수요를 예측할 수 있을까?

월별 관광객 수를 예측 · 코로나 급변기에도 실측에 근접



3.1%

예측 오차 (MAPE)

7.4% → 3.1%

여러 데이터 결합 오차 절반 이하로 감소

데이터

- 파리·암스테르담·리스본 월별 관광객 (2012~2022)
- + 확진자·구글 트렌드·여행 포럼 결합

모델

- Temporal Fusion Transformer (TFT)
- AutoGluon 딥러닝 · 해석 가능

결과

- ARIMA·SVM·LSTM보다 우수
- MAPE 7.4% → 3.1% (데이터 결합)

Interpretable tourism demand forecasting with temporal fusion transformers amid COVID-19, Wu et al., Applied Intelligence 53, 14493–14514 (2023)

AutoML

시계열(Time-series) 데이터 전처리

시계열 — EDA

시계열 EDA — 무엇을 보는가

정형 EDA는 분포를 본다 · 시계열 EDA는 시간 위의 움직임을 본다

관점	정형 EDA	시계열 EDA
기본 단위	각 변수의 분포	시간에 따른 추이
관계	변수 간 상관	계절성 · 주기
결측	결측 개수 (순서 무관)	결측 시점 · 빈 날짜
이상	값의 이상치	급변 구간 (이상 시점)

추이

어디로 가고 있는가?

→ 장기 상승·하락, 수준 변화

주기

반복되는 패턴이 있는가?

→ 요일·월·연 단위 계절성

이상

언제 튀었는가?

→ 급변 구간 · 빈 날짜

시계열 — EDA

시계열 EDA 실습 — tsmode

같은 도구, 시계열 모드 — 정형 EDA와 한 줄만 다르다

```
from data_profiling import ProfileReport
profile = ProfileReport(
    df,
    tsmode=True,          # ← 시계열 모드
    sortby="timestamp",  # 시간 기준 정렬
)
profile.to_file("ts_eda.html")
```

정형 EDA와 차이

tsmode=True, sortby 두 줄뿐

적용 시점

일별 집계 후 · 시계열 형식 변환 전

추이

시간축 그래프

→ 여름 높고 겨울 낮은가

주기

자기상관 · 계절성 탐지

→ 요일·연 주기가 잡히는가

이상

결측 gap 위치

→ 빠진 날짜가 있는가

시계열 — 데이터 형식

시계열 데이터의 형식 — 세 개의 열

AutoGluon은 세 개의 열이 필요 — 어느 것 · 언제 · 얼마

item_id	timestamp	target
seoul_bike	2017-12-01	9,539
seoul_bike	2017-12-02	8,523
seoul_bike	2017-12-03	7,222

item_id

어느 시계열인가?

→ 따릉이는 하나, 모두
seoul_bike

timestamp

언제인가?

→ 날짜 (2017-12-01 ...)

target

얼마인가?

→ 대여량 (9,539 ...)

시계열 예측 — 이론 및 실습

여러 변수를 함께 넣기 — 예측을 더 정확하게

기본 3열에 관련 측정값을 열로 더할 수 있다

item_id	timestamp	target	temperature	humidity	...	seasons
seoul_bike	2017-12-01	9,539	-2.45	45.88	...	Winter
seoul_bike	2017-12-02	8,523	1.32	61.96	...	Winter

파란색(기본 3열)은 그대로, 오른쪽에 **관련 변수 10개**를 열로 덧붙임

과거만 아는 값 (8개)

기온 · 습도 · 풍속 · 가시거리 · 이슬점 · 일사량 · 강수량 · 적설량

"지난 날씨는 알지만, 미래 날씨는 아직 모른다"

미래도 아는 값 (2개)

계절 · 공휴일

"다음 주 토요일이 공휴일인 건 달력만 봐도 안다"

실제 사례 — 관련 변수를 더하면?

관광 수요 예측 — 과거 관광량만 사용 시 오차 **7.4%**

+ 확진자 · 검색 데이터 결합 → 오차 **3.1%** (예측 정확도 2배 이상 향상)

시계열 예측 — 이론 및 실습

집계 방식은 변수마다 다르다

기계는 합계인지 평균인지 모른다

문제 제기 — 시간별 기록 **8,760행**을 일별 **365행**으로 합칠 때
→ 모든 변수를 같은 방식으로 합칠 수 없다

집계 방식	변수	이유
합계 (sum)	대여량 · 강수량 · 적설량 · 일사량	하루 동안 누적된 총량이 의미를 가짐
평균 (mean)	기온 · 습도 · 풍속 · 가시거리 · 이슬점	하루 동안의 대표 상태
첫 값 (first)	계절 · 공휴일 · 운영일	하루 내내 동일한 값

하루 대여량을 평균 내면? → 시간당 평균일 뿐, 그날 몇 대 빌렸는지 알 수 없음
하루 기온을 합계 내면? → 24개를 더한 숫자, 아무 의미 없음

시계열 예측 — 이론 및 실습

시계열 데이터 준비 — 자동과 수동

형식만 맞추면, 나머지는 AutoGluon이 자동 처리

AutoGluon이 자동으로

- **시간 간격 파악**
일별·시간별 여부 자동 판단
- **빠진 시점 채우기**
날짜 누락 시 행 자동 삽입·규칙화
- **결측값 처리**
NaN도 모델이 자체 처리 · 수동 보정 불필요

날짜	대여량
3월 1일	9,539
3월 2일	8,523
3월 3일	NaN ← 자동 삽입
3월 4일	7,222

연구자가 할 것

- **시간 간격 지정**
일별·시간별·월별 (따름이: 일별)
- **결측의 의미 판단**
결측인가 vs "0"인가
→ 수요 예측 시 결측=0이면 0으로 채움

시계열 예측 — 이론 및 실습

따릉이를 시계열 형식으로

원본 14열·시간별 → 시계열 형식 13열·일별

```
# 1) 일별 집계 - 집계 방식은 변수마다 다르게
daily = df.groupby("Date").agg({
    "Rented Bike Count": "sum",      # 합계
    "Temperature(°C)": "mean",      # 평균
    "Seasons": "first",             # 첫 값
    # ... 나머지 변수도 동일하게 지정
}).reset_index()

# 2) 열 이름 정리 + item_id 추가
ts_df = daily.rename(columns={
    "Date": "timestamp",
    "Rented Bike Count": "target",
})
ts_df["item_id"] = "seoul_bike"

# 3) 시계열 형식으로 변환
ts_data = TimeSeriesDataFrame.from_data_frame(
    ts_df,
    id_column="item_id",
    timestamp_column="timestamp",
)
```

► Before — 원본

Date	Hour	Temperature	...	Rented Bike Count
2017-12-01	0	-5.2	...	254
2017-12-01	1	-5.5	...	204

14열 · 8,760행 (365일 × 24시간)



► After — 시계열 형식

item_id	timestamp	target	temperature	...
seoul_bike	2017-12-01	9539	-2.45	...
seoul_bike	2017-12-02	8523	1.32	...

13열 · 365행 · item_id-timestamp = 인덱스

시계열 예측 — 이론 및 실습

빠진 날짜는 자동으로 채워진다

실제 데이터에는 누락된 시점이 존재

```
ts_filled = ts_missing.convert_frequency(freq="D")
```

일 단위(freq="D")만 알려주면 빠진 날짜를 자동으로 채운다

Before — 3~5일 제거 (362행)

timestamp	target	temperature
2017-12-02	8523.0	1.32
2017-12-06	6669.0	1.53

12월 2일 → 곧바로 12월 6일
사이 날짜가 없어 시간이 끊김

After — 3~5일 삽입 (365행)

timestamp	target	temperature
2017-12-02	8523.0	1.32
2017-12-03	NaN	NaN
2017-12-04	NaN	NaN
2017-12-05	NaN	NaN
2017-12-06	6669.0	1.53

빠진 날짜는 행으로 삽입
값은 NaN 그대로

실습 ① — 데이터 준비와 형식 변환

따라 하기 · 따름이

강의에서 본 불러오기 → EDA → 형식 변환 그대로 실행

흐름 익히기

1 · 불러오기

3열 형식(item_id · timestamp · target) 확인

2 · EDA (tsmode)

ProfileReport(df, tsmode=True) 로 추이 · 주기 확인

3 · 형식 변환

TimeSeriesDataFrame · convert_frequency
집계 방식 판단 — 합계? 평균?

AutoML — 전처리

실습 데이터셋

PJM Energy — 지역별 전력 소비량 예측 · 다중 시계열 · 시간별 · 11개 지역
미국 동부 송전망의 지역별 시간당 소비량을 동시에 예측하는 다중 시계열

Steel Industry — 제철 공장 전력 소비 예측 · 단일 시계열 · 15분 · 1년
전남 광양 제철소의 15분 단위 전력량으로 집계·공변량을 다루는 국내 데이터

Berkeley Earth — 도시별 기온 추세 · 다중 시계열 · 월별 · ~270년
서울 등 세계 100개 도시의 월평균 기온으로 장기 추세와 계절성을 분해

CDC NHSN — 병원 호흡기 입원 예측 · 다중 시계열 · 주별
미국 병원의 주간 독감·코로나 입원 수로 병상 수요를 예측하는 공중보건 데이터

AutoML — 전처리

실습 데이터셋

CDC NWSS — 하수 바이러스 활동 예측 · 다중 시계열 · 주별 · 40개 지점
하수에서 검출된 바이러스량으로 임상 검사보다 먼저 유행을 탐지하는 조기 경보

Melbourne Pedestrian — 도시 보행자 통행량 예측 · 다중 시계열 · 일별 · 10개 센서
멜버른 센서별 보행자 수로, COVID 봉쇄 급락이 담긴 구조 단절 사례

Saugeen River — 하천 유량 예측 · 단일 시계열 · 일별 · 65년
캐나다 소지강 일평균 유량으로 치우친 분포와 장기 계절성을 다루는 최다 정제 데이터

AutoML

시계열(Time-series) 데이터 모델 학습

AutoGluon 시계열이 학습하는 모델들

정형 데이터와 같은 구조 — 시계열용 모델로 교체

통계 모델

ETS · ARIMA · Theta · SeasonalNaive
추세·계절성을 수식으로 포착하는 전통 기법

지역

트리 기반

RecursiveTabular · DirectTabular
시계열을 회귀 문제로 변환 → LightGBM 사용

전역

딥러닝

DeepAR · TemporalFusionTransformer · PatchTST
복잡한 패턴을 신경망으로 학습

전역

파운데이션

Chronos
대량의 시계열로 미리 학습되어 별도 학습 없이 예측

전역

사전학습

시계열 예측 — 모델 학습

지역 모델

지역 모델 (Local)

시계열 하나마다 모델을 따로 학습

예: ARIMA · ETS · Theta · SeasonalNaive

- ✓ 데이터가 적어도 잘 작동
- ✓ 해석이 쉬움 (수식 기반)
- ✗ 유연성이 낮음 (복잡한 패턴 포착 어려움)
- ✗ 예측이 느림 (새 시계열마다 처음부터 학습)



시계열 예측 — 모델 학습

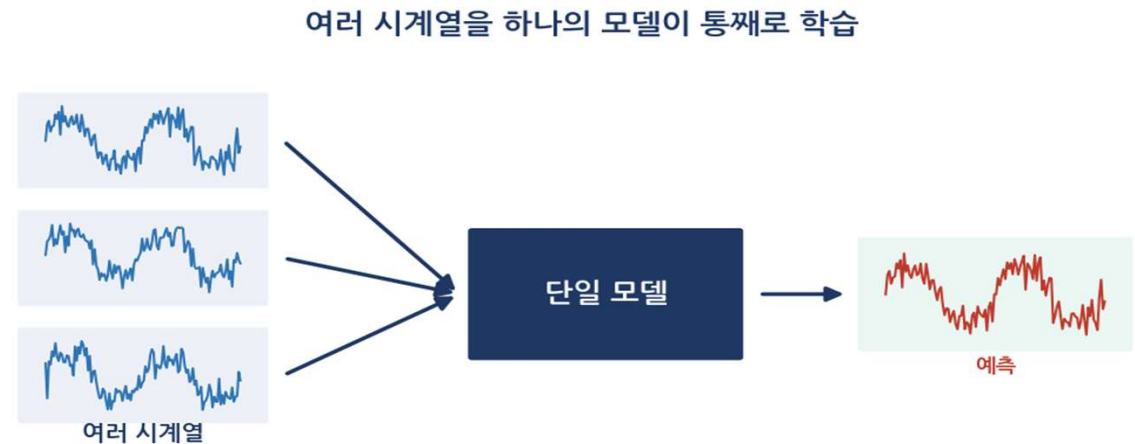
전역 모델

전역 모델 (Global)

여러 시계열을 하나의 모델이 통째로 학습

예: LightGBM 기반(RecursiveTabular) · DeepAR · TFT · PatchTST

- ✓ 유연성이 높음 (복잡한 패턴 학습)
- ✓ 예측이 빠름 (학습된 모델 재사용)
- ✗ 학습이 느림
- ✗ 데이터가 많이 필요함



시계열 예측 — 모델 학습

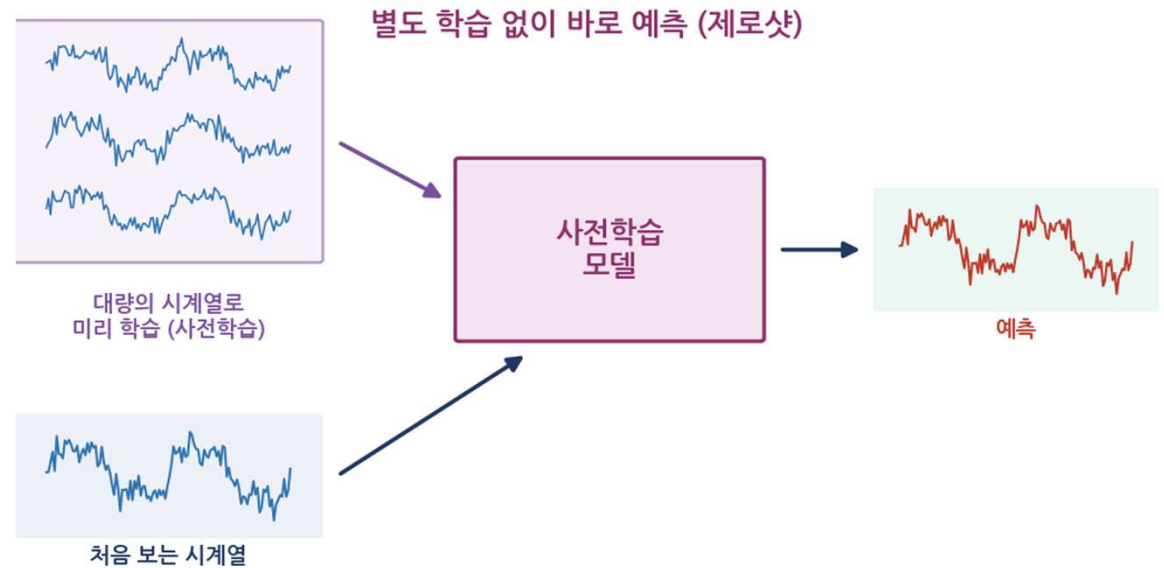
사전학습 모델

사전학습 모델 (Pretrained)

대량의 시계열로 이미 학습되어 있음

예: Chronos

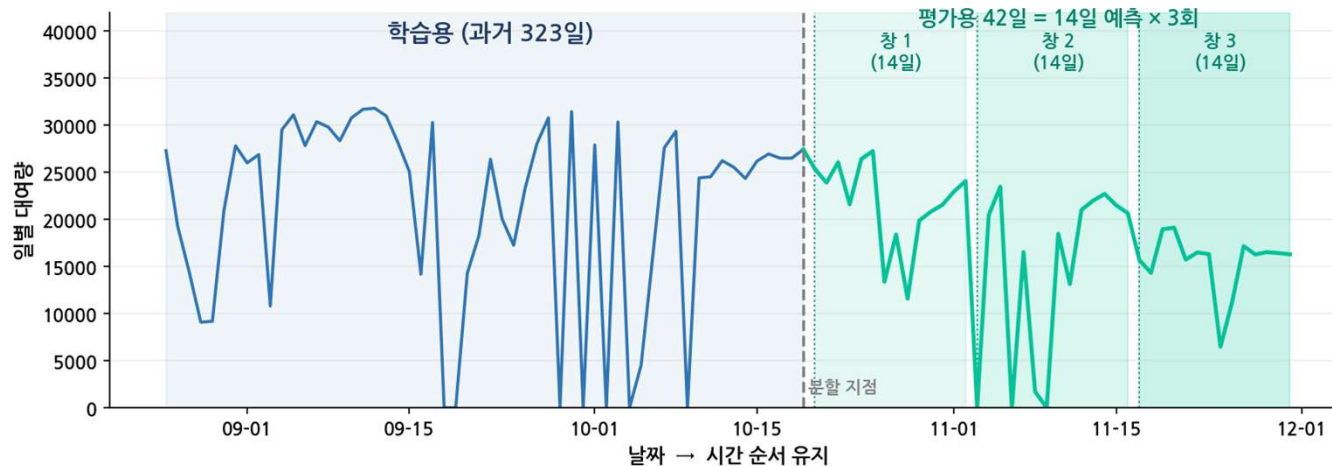
- ✓ 별도 학습 없이 바로 예측 (제로샷)
- ✓ 처음 보는 데이터에도 작동



시계열 예측 — 모델 학습

학습과 평가 분할 — 과거로 배우고, 미래로 시험한다

시계열은 무작위로 나누면 안 된다



왜 무작위로 나누면 안 되나

- 정형: 무작위 80/20 분할 — 행이 독립적, 순서 무관
- 시계열: 무작위로 섞으면 안 됨
 - 미래를 본 채 과거를 맞추는 것
 - 성능이 실제보다 부풀려짐

prediction_length = 14일

평가용 = 14 × 3 = 42일

```
prediction_length = 14
num_test_windows = 3
train_data, test_data = ts_data.train_test_split(
    num_test_windows * prediction_length) # 42일
```

시계열 예측 — 모델 학습

하나의 시계열로 어떻게 학습하나 — 창을 옮겨가며

정형: 8,760행 = 8,760개 학습 사례 VS 시계열: 365일짜리 딱 하나
→ 학습 사례가 1개뿐인데, 어떻게 학습할까?



창을 옮겨가며 학습 문제를 여러 개 생성
→ 323일에서 수백 개의 학습 사례 확보
각 사례 = "과거를 보고 다음 14일을 맞추라"

하나의 시계열 → 창을 옮겨가며 수백 개의 학습 사례

시계열 예측 — 모델 학습

시계열 모델 학습 fit()

시간 순 분할

```
prediction_length = 14 # 며칠 앞을 예측할 것인가
num_test_windows = 3  # 몇 번 시험 볼 것인가

train_data, test_data = ts_data.train_test_split(
    num_test_windows * prediction_length # 42일
)
```

모델 학습

```
predictor = TimeSeriesPredictor(
    prediction_length=14, # 14일 앞 예측
    target="target",      # 예측 대상 = 대여량
    freq="D",             # 일별 데이터
).fit(
    train_data,
    presets="medium_quality", # 성능·시간 균형
    time_limit=600,          # 최대 10분
)
```

정형 데이터의 TabularPredictor와 같은 구조

시간 순 분할 — 미래 데이터가 훈련에 유출되지 않도록 순서대로 나눔

- 검증 구간 = num_test_windows × prediction_length = 42일
- presets · time_limit — 성능과 시간의 균형을 맞춤

시계열 예측 — 모델 학습

모델 성능 비교 leaderboard()

코드

```
predictor.leaderboard(test_data)
```

출력

model	score_test	score_val	fit_time
Chronos2	-0.1441	-0.2204	1.26
DynamicOptimizedTheta	-0.1764	-0.4337	0.01
Chronos2SmallFineTuned	-0.1783	-0.2113	85.48
AutoETS	-0.1802	-0.4058	0.01
WeightedEnsemble	-0.1896	-0.1407	0.79
DeepAR	-0.2418	-0.1590	45.50
SeasonalNaive	-0.2607	-0.5191	0.02
ChronosWithRegressor[bolt_small]	-0.2626	-0.1837	0.34
TemporalFusionTransformer	-0.2670	-0.1571	54.30
DirectTabular	-0.3233	-0.2106	6.22
RecursiveTabular	-0.6010	-0.4867	7.92

최상단이 1위 (score_test 기준)

- score_test — 평가용 성능, 0에 가까울수록 우수
- 1위 = Chronos2 (-0.1441), 사전학습 시계열 모델
- '클수록 우수'로 부호 통일 → 점수에 - 부착

앙상블이 늘 1위는 아니다

- WeightedEnsemble은 5위 (-0.1896)
- 강력한 단일 모델(Chronos2)이 앞설 수 있음

열 읽는 법

- score_val — 검증 성능 (모델 선택 기준)
- fit_time — 학습 시간(초)

시계열 예측 — 모델 학습

예측 predict()

코드

```
predictions = predictor.predict(train_data)

predictions.head()
```

출력

timestamp	mean	0.1	0.5	0.9
2018-11-17	18,858	8,962	18,858	22,339
2018-11-18	17,651	5,638	17,651	21,877
2018-11-19	17,611	5,467	17,611	21,884

정형 회귀와의 차이

- 정형: 예측값 하나
- 시계열: 평균 + 범위(분위수)

열 의미

- mean — 평균 예측(기댓값)
- 0.1 = P10 — 10%가 이보다 낮음
- 0.5 = P50 — 중앙값
- 0.9 = P90 — 90%가 이보다 낮음
- P10 ~ P90 = 80% 예측 구간
- 예: 11/17 평균 18,858대 · 80% 구간 8,962~22,339대

왜 범위인가

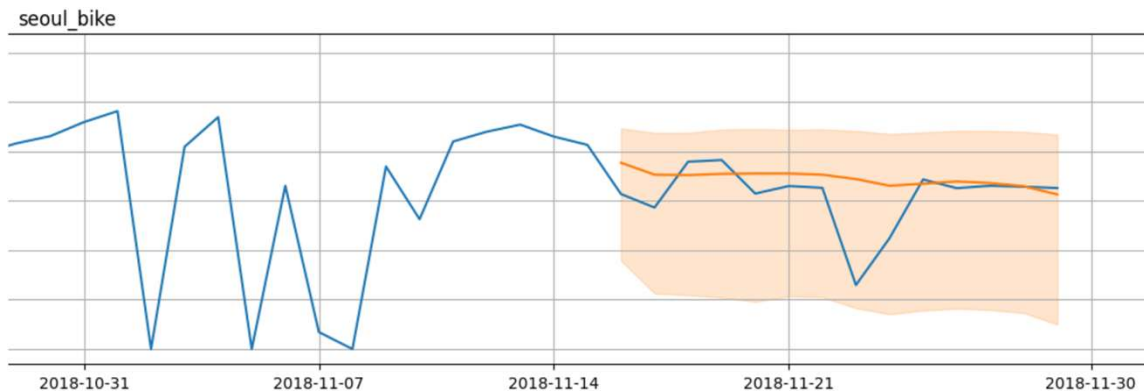
- 미래는 불확실 → 단일 값은 반드시 틀림
- 범위가 있어야 의사결정 가능

시계열 예측 — 모델 학습

예측 결과 시각화 plot()

코드

```
predictor.plot(  
  data=test_data,  
  predictions=predictions,  
  quantile_levels=[0.1, 0.9], # P10 ~ P90 표시  
  max_history_length=60,      # 과거 60일만  
);
```



그래프 구성

- 파란 선 — 과거 관측값 + 실제값 (Observed)
- 주황 선 — 평균 예측 (Forecast)
- 음영 영역 — P10 ~ P90 (80% 예측 범위)

확인할 점

- 예측선이 실제 흐름을 따라가는가?
- 실제값이 음영 안에 들어오는가?
- 음영이 오른쪽으로 갈수록 넓어지는가? (미래로 갈수록 불확실)

벗어난 날이 있다면

- 그날 무슨 일이 있었는가?
- 예: 2018-11-24 폭설 (적설 78cm)
- 대여량 16,314 → 6,477대로 급락
- 모델은 과거 대여량만 봤을 뿐, 눈이 올 줄 몰랐다

시계열 예측 — 모델 학습

연구자가 조절하는 학습 설정

```
predictor = TimeSeriesPredictor(  
    prediction_length=14,  
    target="target",  
    freq="D",  
    eval_metric="RMSE",          # 평가 기준 변경  
)  
.fit(  
    train_data,  
    presets="fast_training",  
    time_limit=180,
```

설정 표

설정	역할	값의 예
prediction_length	며칠 앞을 예측	14 (2주)
known_covariates_names	미래를 아는 변수	["seasons", "holiday"]
eval_metric	성능 판단 기준	WQL(기본) · RMSE · MASE
presets	성능·시간 균형	fast_training / medium_quality / high_quality / best_quality
time_limit	최대 학습 시간(초)	600 (10분)

시계열 예측 — 모델 학습

변수 중요도 feature_importance()

코드

```
predictor.feature_importance(  
    data=test_data,  
    time_limit=300,  
)
```

출력

feature	importance
functioning_day	0.0323
wind_speed	0.0162
rainfall	0.0061
humidity	0.0048
dew_point	0.0014
visibility	0.0013
temperature	-0.0018
solar_radiation	-0.0035

정형 데이터와 같은 원리

- 순열 중요도 — 값을 뒤섞어 성능 하락폭 측정
- 크게 떨어질수록 중요한 변수

확인할 점

- 날씨 변수가 상위에 오르는가?
- 값이 0에 가깝다 → 모델이 안 쓰는 변수

모델마다 쓰는 변수

모델	과거만 아는 값 (기온·강수량)
통계 (ETS·SeasonalNaive)	❌ 안 씀
트리 (RecursiveTabular)	❌ 안 씀
TFT · Chronos2	✅ 사용

실습 ② — 학습과 해석

데이터 선택

- 앞의 실습 데이터셋 중 하나 선택
- 실습①의 준비 · 변환 과정을 그 데이터에 적용

학습 · 해석

4 · 학습

`TimeSeriesPredictor(prediction_length=...).fit(train) · leaderboard`

5 · 해석

`predict · plot` 으로 예측 시각화
예측 구간(P10~P90) 확인